STI V2.0 BY A'e

LE MANUEL CONNECTÉ

LYCÉE RAFÈHA

4ème SI

STI

Systèmes et technologies de l'informatique

HTML5 / CSS3 / JAVASCRIPT

BASES DE DONNÉES / PHP

Cours • Méthodes • Travaux pratiques

77 fiches • 18 pages de tableaux récapitulatifs

HTML5 CSS3 JavaScript SQL / MySQL PHP

Enseignant

Aymen Essouyah

SMART BOOK

Lire. Scanner. Pratiquer.

SCANNER POUR APPRENDRE

ÉDITION CONNECTÉE · 2026

COURS · EXERCICES · RÉVISIONS

Sommaire

MODE D’EMPLOI DU LIVRE CONNECTÉ

Sur écran : cliquez sur le titre du sommaire ou sur « Ouvrir sur STI V2.0 BY A'e ». Sur papier : scannez le QR contrasté au bas de la fiche pour ouvrir la partie correspondante. Une connexion Internet est nécessaire.

Depuis une photo enregistrée : utilisez la détection de QR de votre téléphone. La reconnaissance automatique du texte d’une page, sans QR lisible, n’est pas intégrée. Le filigrane pâle est une signature discrète.

HTML5

01. Structure d’un document HTML5 ··· 6

02. Syntaxe, attributs et imbrication ··· 7

03. Texte, liens et contenus multimédias ··· 8

04. Listes, tableaux et structure sémantique ··· 9

05. Formulaires : saisie et transmission ··· 10

06. Formulaires : options et aide-mémoire ··· 11

07. Cadres intégrés et cibles de navigation ··· 12

08. Attributs globaux, datalist et événements HTML ··· 13

CSS3

09. Syntaxe et intégration des styles ··· 14

10. Sélecteurs, classes et identifiants ··· 15

11. Unités et modèle de boîte ··· 16

12. Texte, couleurs, bordures et arrière-plans ··· 17

13. Positionnement : relative et absolute ··· 18

14. Positionnement : fixed et sticky ··· 19

15. Mise en page avec float et clear ··· 20

16. Flexbox, affichage et débordement ··· 21

17. Liens, listes, tableaux et ombres ··· 22

18. Filtres et transformations 2D ··· 23

19. Transitions et animations ··· 24

JavaScript

20. Intégration, variables et types ··· 25

Sommaire — suite

JavaScript

21. Opérateurs, conversions et dialogues ··· 26
22. Conditions et choix multiples ··· 27
23. Boucles et accumulation ··· 28
24. TP : jeu de devinettes ··· 29
25. DOM : sélectionner et modifier des éléments ··· 30
26. DOM : lire et contrôler un formulaire ··· 31
27. Fonctions, portée et événements ··· 32
28. Nombres, chaînes et méthodes usuelles ··· 33
29. Tableaux et parcours ··· 34
30. Dates, temporisation et horloge numérique ··· 35
31. Compléments : Math, chaînes et sorties ··· 36
32. Médias, styles et dates de formulaire ··· 37

Bases de données et SQL

33. Notions relationnelles et contraintes ··· 38
34. SQL : commandes et types MySQL ··· 39
35. Création des tables : gestion des locations ··· 40
36. Modifier et supprimer une structure ··· 41
37. Insérer, modifier et supprimer des lignes ··· 42
38. Base commerciale : schéma des exercices ··· 43
39. SELECT, filtres et expressions ··· 44
40. Agrégats, tri et sous-requêtes ··· 45

Sommaire — suite

Bases de données et SQL

41. Construire une requête SQL : la méthode ··· 46
42. Construire une requête SQL : cas particuliers ··· 47
43. GROUP BY, HAVING et jointures ··· 48
44. Fonctions SQL sur les chaînes ··· 49
45. Fonctions SQL sur les dates et heures ··· 50
46. Jointures dans WHERE et mises à jour croisées ··· 51

PHP

47. Principe client-serveur et environnement ··· 52
48. Variables, types, constantes et opérateurs ··· 53
49. Formulaires : GET, POST, isset et empty ··· 54
50. Conditions, boucles et fonctions ··· 55
51. Tableaux, parcours avec for et tris ··· 56
52. Chaînes et fonctions usuelles ··· 57
53. Dates et heures : timestamp et checkdate ··· 58
54. Dates et heures : date et getdate ··· 59
55. Dates et heures : time et mktime ··· 60
56. Dates et heures : strtotime et calculs ··· 61
57. MySQLi procédural : connexion et requête fixe ··· 62
58. include, require et organisation des fichiers ··· 63
59. MySQLi : SELECT, INSERT, UPDATE et DELETE ··· 64
60. Conversions, guillemets et fonctions arithmétiques ··· 65

Sommaire — suite

PHP

61. MySQLi : lecture des résultats et saisie numérique ··· 66

62. Niveaux d’erreur PHP : tableau et exemples ··· 67

63. Comprendre les erreurs PHP et les diagnostics ··· 68

Travaux pratiques

64. Projet 1 : gestion des élèves — énoncé et base ··· 69

65. Projet 1 : validation par caractères ··· 70

66. Projet 1 : formulaire et validation JavaScript ··· 71

67. Projet 1 : connexion et insertion pédagogique ··· 72

68. Projet 1 : lecture, modification et statistiques ··· 73

69. Projet 2 : recharge électrique — énoncé et base ··· 74

70. Projet 2 : horaires et insertion pédagogique ··· 75

71. Projet 2 : statistiques, coûts et tests ··· 76

Fiches méthode

72. Méthode de résolution et lecture de programme ··· 77

73. Erreurs fréquentes et réflexes de correction ··· 78

74. Tests, vérification et checklist de remise ··· 79

Annexes

75. Synthèse et contrôle des acquis ··· 80

76. Révision : questions et réponses essentielles ··· 81

77. Sources, périmètre et conventions de cette édition ··· 82

Tableaux récapitulatifs de fin de livre

HTML5 — balises et attributs ··· 83

CSS3 — propriétés par thèmes ··· 87

JavaScript — syntaxe et API ··· 91

Bases de données et SQL — LDD / LMD ··· 95

PHP — fonctions : syntaxe et description ··· 97

Structure d’un document HTML5

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Objectif

Créer un document HTML valide et comprendre le rôle de ses éléments principaux. HTML structure le contenu ; CSS gère sa présentation ; JavaScript ajoute des interactions.

Document de départ

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <meta name="viewport"
        content="width=device-width, initial-scale=1">
  <meta name="description" content="Cours de STI">
  <title>Ma première page</title>
  <link rel="stylesheet" href="style.css">
  <script src="script.js" defer></script>
</head>
<body>
  <h1>Bienvenue au Lycée Rafèha</h1>
  <p>Une page structurée en HTML5.</p>
</body>
</html>
```

Rôle des éléments

• <!DOCTYPE html> active le mode standard du navigateur.

• html est l’élément racine. lang="fr" indique la langue du contenu.

• head contient le titre et les métadonnées. title est affiché dans l’onglet du navigateur.

• meta charset="utf-8" définit l’encodage ; link relie la feuille de style.

• body contient le contenu de la page. Les couleurs et les images de fond se règlent en CSS, pas avec l’ancien attribut background.

• defer reporte l’exécution du script externe après l’analyse du HTML.

À réaliser

Enregistrer ce document sous index.html, créer style.css et script.js dans le même dossier, puis ouvrir la page dans le navigateur.

Syntaxe, attributs et imbrication

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Éléments et attributs

Une balise ouvrante peut porter des attributs. Les éléments non vides ont une balise fermante. Respecter l'imbrication : le dernier élément ouvert est le premier fermé.

```
<p class="important" id="intro" title="Présentation">
  Un texte <strong>important</strong>.
</p>
<a href="contact.html">Nous contacter</a>
<img src="lycee.jpg" alt="Entrée du lycée" width="400">
```

- id identifie un seul élément dans le document.
- class contient un ou plusieurs noms de classes séparés par des espaces.
- title apporte une information complémentaire ; il ne remplace pas un libellé accessible.
- style permet un style local ; privilégier une feuille CSS externe.

Éléments vides

br, hr, img, input, meta, link et source n'ont pas de balise fermante en HTML. Le slash final de
 est facultatif et ne transforme pas l'élément en paire de balises.

Casse et caractères réservés

Les noms des balises et des attributs HTML ne distinguent généralement pas les majuscules des minuscules. En revanche, les valeurs de id et de class doivent conserver leur casse pour les sélecteurs CSS et les accès JavaScript.

```
<p>Écrire &lt;p&gt; pour afficher le nom de la balise.</p>
<p>Utiliser &amp; pour afficher une esperluette.</p>
<!-- Ceci est un commentaire HTML -->
```

Bonnes pratiques

- Employer des noms simples : menu-principal, titre, carte.
- Indenter le code et entourer les valeurs d'attributs de guillemets droits.
- Ne pas imbriquer un paragraphe dans un autre paragraphe.
- Préférer les éléments sémantiques aux éléments génériques lorsque leur rôle correspond au contenu.

Application

Créer deux paragraphes de même classe et leur attribuer deux identifiants différents. Expliquer pourquoi deux éléments ne doivent pas partager le même id.

Texte, liens et contenus multimédias

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Structurer le texte

h1 à h6 représentent des titres hiérarchisés ; p un paragraphe ; br un saut de ligne ; hr une séparation thématique. Choisir un titre selon son niveau, et non selon sa taille visuelle.

• strong : importance forte ; em : emphase ; mark : passage mis en évidence.

• abbr : abréviation ; sup et sub : exposant et indice.

• blockquote : citation en bloc ; q : citation courte ; cite : titre d’une œuvre.

• del et ins : contenu supprimé ou ajouté ; pre et code : code préformaté.

• address : coordonnées de contact ; time : date ou heure.

```
<h1>Le Web</h1>
<h2>Les langages</h2>
<p><abbr title="HyperText Markup Language">HTML</abbr>
structure la page. CSS gère sa <em>présentation</em>.</p>
<p>H<sub>2</sub>O ; x<sup>2</sup></p>
<time datetime="2026-10-21">21 octobre 2026</time>
<a href="https://www.php.net/">Documentation PHP</a>
<a href="#suite">Aller à la suite</a>
<h2 id="suite">Suite du cours</h2>
```

Images, audio et vidéo

```
<figure>
  <img src="schema.png" alt="Échanges client-serveur">
  <figcaption>Le navigateur dialogue avec le serveur.</figcaption>
</figure>
<audio controls>
  <source src="cours.mp3" type="audio/mpeg">
  Votre navigateur ne peut pas lire ce fichier audio.
</audio>
<video controls width="480">
  <source src="demo.mp4" type="video/mp4">
  Votre navigateur ne peut pas lire cette vidéo.
</video>
```

À retenir

alt décrit une image informative ; alt="" convient à une image purement décorative. Pour une vidéo pédagogique, prévoir des sous-titres. Un lien doit avoir un intitulé compréhensible hors contexte.

Listes, tableaux et structure sémantique

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Les listes

```
<ul><li>HTML5</li><li>CSS3</li></ul>
<ol><li>Créer la page</li><li>Tester</li></ol>
<dl>
  <dt>DOM</dt><dd>Représentation de la page en objets.</dd>
</dl>
```

ul crée une liste non ordonnée ; ol une liste ordonnée ; li un élément de liste. dl regroupe des termes dt et leurs descriptions dd.

Les tableaux de données

```
<table>
  <caption>Résultats des élèves</caption>
  <thead>
    <tr><th scope="col">Nom</th><th scope="col">Note</th></tr>
  </thead>
  <tbody>
    <tr><td>Amal</td><td>16</td></tr>
    <tr><td>Yassine</td><td>14</td></tr>
  </tbody>
</table>
```

tr définit une ligne ; th une cellule d’en-tête ; td une cellule de données. thead, tbody et tfoot regroupent les lignes. colspan et rowspan permettent des fusions de cellules. Ne pas utiliser un tableau pour mettre en page tout le site.

Organiser une page

```
<header><h1>Mon site</h1></header>
<nav><a href="index.html">Accueil</a></nav>
<main>
  <section>
    <h2>Actualités</h2>
    <article><h3>Projet STI</h3><p>Présentation...</p></article>
  </section>
  <aside>Informations complémentaires</aside>
</main>
<footer>Contact du lycée</footer>
```

header : en-tête ; nav : navigation ; main : contenu principal ; section : section thématique ; article : contenu autonome ; aside : complément ; footer : pied de page.

Éléments génériques

Par défaut, div est affiché en bloc et span en ligne. Ils servent à regrouper du contenu sans lui attribuer de signification sémantique particulière. Ils restent utiles même sans attribut CSS.

Formulaires : saisie et transmission

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Formulaire complet

```
<form id="inscription" method="post" action="inscrire.php">
  <fieldset>
    <legend>Identification</legend>
    <label for="nom">Nom</label>
    <input type="text" id="nom" name="nom"
           maxlength="50" required>
    <label for="email">Adresse électronique</label>
    <input type="email" id="email" name="email" required>
    <label for="naissance">Date de naissance</label>
    <input type="date" id="naissance" name="naissance">
    <label for="classe">Classe</label>
    <select id="classe" name="classe">
      <option value="4SI1">4ème SI 1</option>
      <option value="4SI2">4ème SI 2</option>
    </select>
    <label for="message">Message</label>
    <textarea id="message" name="message" rows="4"></textarea>
    <button type="submit">Envoyer</button>
    <button type="reset">Réinitialiser</button>
  </fieldset>
</form>
```

Comprendre les attributs

• action désigne l'URL de traitement ; method choisit get ou post. Leur présence explicite améliore la lisibilité, sans être toujours obligatoire en HTML.

• name est la clé utilisée pour transmettre la valeur au serveur ; id sert notamment à l'association avec label for.

• required rend la saisie obligatoire côté navigateur ; maxlength limite la longueur ; placeholder donne une indication, sans remplacer label.

• min, max et step règlent les bornes et le pas d'un champ number.

• readonly interdit la modification mais conserve l'envoi ; disabled désactive le champ et l'exclut de l'envoi.

GET ou POST ?

GET place les paramètres dans l'URL et convient notamment aux recherches. POST transmet les données dans le corps de la requête et convient aux modifications. POST ne chiffre pas les données : utiliser HTTPS.

Important

Les validations HTML et JavaScript facilitent la saisie mais ne remplacent jamais la validation sur le serveur PHP.

Formulaires : options et aide-mémoire

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Cases à cocher et boutons radio

```
<fieldset>
  <legend>Langages étudiés</legend>
  <label><input type="checkbox" name="langages[]"
                value="html" checked> HTML</label>
  <label><input type="checkbox" name="langages[]"
                value="php"> PHP</label>
</fieldset>
<fieldset>
  <legend>Groupe</legend>
  <label><input type="radio" name="groupe"
                value="A" required> A</label>
  <label><input type="radio" name="groupe" value="B"> B</label>
</fieldset>
```

Les radios de même name sont exclusifs. Les cases à cocher autorisent plusieurs choix ; elles peuvent partager un nom de tableau, par exemple langages[], pour leur traitement en PHP. Une case non cochée n'est normalement pas transmise.

Liste multiple et groupes

```
<select name="options[]" multiple size="4">
  <optgroup label="Web">
    <option value="html" selected>HTML</option>
    <option value="css">CSS</option>
  </optgroup>
</select>
<input type="tel" name="telephone" pattern="[0-9]{8}">
<input type="number" name="note" min="0" max="20" step="0.5">
```

Autres types de champs

- password : saisie masquée, mais non chiffrée par ce seul attribut.
- search : recherche ; url : adresse Web ; tel : téléphone.
- range : curseur ; color : couleur ; date : date ; file : fichier.
- hidden : valeur invisible dans l'interface, mais modifiable par l'utilisateur.

Envoi de fichier

Utiliser method="post" et enctype="multipart/form-data" sur form, ainsi qu'un input type="file". PHP récupère le fichier via $_FILES. Vérifier côté serveur sa taille, son type réel et son nom de destination.

Application

Construire un formulaire avec une adresse électronique, une note de 0 à 20, un groupe et plusieurs langages. Vérifier les données transmises dans les outils réseau du navigateur.

Cadres intégrés et cibles de navigation

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

iframe : intégrer un document

iframe affiche une page dans une autre. title fournit un nom accessible au cadre ; name permet de le désigner comme cible. id et name n'ont pas le même rôle.

```
<nav>
  <a href="inscription.html" target="cadre">Inscription</a>
  <a href="liste.php" target="cadre">Liste des élèves</a>
</nav>
<iframe name="cadre" id="cadre" title="Espace de travail"
        src="inscription.html"></iframe>
```

```
iframe { width: 100%; min-height: 500px; border: 0; }
```

target sur les liens et formulaires

- _self : contexte courant ; valeur usuelle par défaut.
- _blank : nouveau contexte, généralement un nouvel onglet.
- _parent : contexte parent ; _top : contexte de navigation supérieur.
- cadre : contexte portant ce nom, ici l'iframe de la page.

```
<form action="ajouter.php" method="post" target="cadre">
  <label for="nom">Nom</label>
  <input id="nom" name="nom" required>
  <button type="submit">Ajouter</button>
</form>
<a href="https://www.php.net/" target="_blank"
   rel="noopener">Documentation PHP</a>
```

Si le formulaire est déjà affiché dans cadre, _self permet aussi d'afficher sa réponse dans ce cadre. Si aucun contexte nommé ne correspond à une cible, le navigateur peut en ouvrir un nouveau.

Sécurité et limites

Une page tierce peut interdire son intégration par des en-têtes de sécurité. La politique de même origine limite les accès JavaScript entre documents de domaines différents. sandbox permet de restreindre un cadre ; ses permissions doivent être choisies selon le besoin. Ne pas ajouter toutes les permissions sans les comprendre.

Application

Créer index.html avec une navigation et un cadre nommé cadre. Afficher successivement deux pages locales. Tester ensuite un formulaire soumis vers ajouter.php sur un serveur PHP.

Attributs globaux, datalist et événements HTML

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Attributs globaux utiles

id identifie ; class regroupe ; title apporte une information ; lang indique une langue ; hidden masque un contenu ; data-* stocke des données propres à l'application. tabindex influence la navigation au clavier : éviter les valeurs positives pour ne pas perturber l'ordre naturel.

```
<article class="produit" data-code="P01" lang="fr">
  <h2>Un produit</h2>
</article>
<label for="ville">Ville</label>
<input id="ville" name="ville" list="villes">
<datalist id="villes">
  <option value="Tunis"></option>
  <option value="Sfax"></option>
  <option value="Sousse"></option>
</datalist>
```

datalist propose des suggestions sans interdire une autre saisie. select propose une liste de choix. Pour imposer une ville reconnue, contrôler sa valeur côté serveur.

Événements dans les attributs

```
<button type="button" onclick="saluer()">Bonjour</button>
<form action="traitement.php" method="post"
      onsubmit="return valider()">
  <input name="nom" required>
  <button type="submit">Envoyer</button>
</form>
```

Dans cet exemple, le return de onsubmit transmet le résultat de valider : false annule l'action par défaut. Écrire seulement onsubmit="valider()" ne transmet pas automatiquement ce false. Pour ce cours, utiliser onsubmit="return valider()" et faire renvoyer false par la fonction lorsque l'envoi doit être bloqué.

Correspondance des noms

onclick correspond à click ; onchange à change ; oninput à input ; onsubmit à submit. input réagit au changement pendant la saisie ; change intervient lors de la validation d'un changement selon le contrôle. Éviter de multiplier les gestionnaires dans le HTML : addEventListener sépare mieux structure et comportement.

Application

Construire une zone de saisie avec suggestions et un message de validation. Conserver un libellé visible ; ne pas faire dépendre les consignes uniquement de title ou de placeholder.

Syntaxe et intégration des styles

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Définition

CSS (Cascading Style Sheets) décrit la présentation des éléments HTML. Une règle associe un sélecteur à des déclarations propriété: valeur.

```
p {
  color: #16324f;
  font-size: 16px;
  line-height: 1.6;
}
```

Trois modes d’intégration

```
<!-- Feuille externe : méthode à privilégier -->
<link rel="stylesheet" href="style.css">
<!-- Style interne, dans head -->
<style>
  h1 { color: navy; }
</style>
<!-- Style local -->
<p style="color: navy;">Un paragraphe</p>
```

Cascade, héritage et priorité

Plusieurs règles peuvent viser un même élément. Pour les déclarations ordinaires d’une même origine et d’une même couche, la spécificité du sélecteur puis l’ordre d’écriture départagent les règles concurrentes. Un identifiant est plus spécifique qu’une classe, qui est plus spécifique qu’un nom de balise.

Certaines propriétés sont héritées, comme color et font-family. Les marges et les bordures ne le sont généralement pas. Éviter l’emploi systématique de !important.

```
body { font-family: Arial, sans-serif; color: #222; }
p { color: blue; }
.important { color: darkred; }
#intro { color: darkgreen; }
/* Un p ayant class="important" et id="intro" sera vert. */
```

Règles de syntaxe

• Terminer chaque déclaration par un point-virgule.

• Ne pas séparer la valeur et son unité : 150px, et non 150 px.

• Écrire les commentaires entre /* et */.

• Une règle CSS ordinaire commence par un sélecteur ; les règles commençant par @, comme @media, suivent une syntaxe particulière.

Application

Relier une feuille externe à la page d’accueil et définir les styles du corps, des titres et des paragraphes.

Sélecteurs, classes et identifiants

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Principaux sélecteurs

• p : tous les paragraphes ; * : tous les éléments.

• .important : les éléments portant la classe important.

• #intro : l'élément portant l'identifiant intro.

• h1, h2 : réunion de plusieurs sélecteurs.

• nav a : liens descendants de nav ; nav > a : liens enfants directs.

• input[type="email"] : champs ayant cet attribut et cette valeur.

• a:hover : lien survolé ; input:focus : champ ayant le focus.

```
<h1 id="titre" class="grand souligne">Le Web</h1>
<p class="important">À retenir</p>
<p class="grand important">Un autre paragraphe</p>
```

```
#titre { color: #16324f; }
.grand { font-size: 24px; }
.souligne { text-decoration: underline; }
.important { font-weight: bold; }
p.important { border-left: 3px solid teal; }
a:hover { color: darkred; }
input:focus { outline: 2px solid #167d8d; }
```

Bien choisir id et class

Un id doit être unique dans le document. Une classe est réutilisable et un élément peut porter plusieurs classes. Pour des noms faciles à manipuler dans ce cours, utiliser des lettres minuscules, des chiffres après la première lettre et des tirets.

Les valeurs de class et de id sont sensibles à la casse : .important ne désigne pas class="Important". Les règles complètes des identifiants CSS autorisent davantage de caractères que cette convention simplifiée.

Sélecteurs combinés

```
.carte .titre { color: navy; } /* descendant */
.carte.titre { color: teal; } /* les deux classes */
li:first-child { font-weight: bold; }
tr:nth-child(even) { background-color: #eef4f7; }
```

Application

Créer trois cartes partageant la classe carte. Ajouter une classe selectionnee à une seule carte et lui donner une bordure différente.

Unités et modèle de boîte

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Unités absolues et relatives

• px : pixel CSS ; pt : point ; cm, mm et in : unités physiques.

• em : taille de police de l'élément ; pour font-size, référence à la taille héritée.

• % : référence dépendant de la propriété, souvent une dimension du conteneur.

• vw et vh : centième de la largeur ou de la hauteur de la zone d'affichage.

• ch : mesure liée à la largeur du caractère zéro ; ex : hauteur d'x de la police.

16px est une taille courante par défaut, pas une égalité universelle pour 1em.

Les quatre zones d'une boîte

Du centre vers l'extérieur : contenu, padding, border, margin.

Avec box-sizing: content-box, width désigne la largeur du contenu. La largeur extérieure occupée, hors cas particuliers de mise en page, vaut :

Largeur = width + padding gauche/droit + border gauche/droit + margin gauche/droit.

```
.boite {
  width: 200px;
  padding: 10px;
  border: 2px solid teal;
  margin: 15px;
}
/* Boîte bordure comprise : 224px ; avec marges : 254px. */
```

Le modèle border-box

```
* { box-sizing: border-box; }
.boite {
  width: 200px;
  padding: 10px;
  border: 2px solid teal;
  margin: 15px;
}
/* Boîte bordure comprise : 200px ; avec marges : 230px. */
```

Notation courte

margin: 1em 0 2em 0.5em suit l'ordre haut, droite, bas, gauche. Deux valeurs définissent les côtés verticaux puis horizontaux. Une valeur s'applique aux quatre côtés. Les mêmes règles valent pour padding.

Attention

Certaines marges verticales de blocs peuvent fusionner. Pour une boîte en ligne non remplacée comme span, width et height ne s'appliquent pas comme sur un bloc ; utiliser inline-block si nécessaire.

Texte, couleurs, bordures et arrière-plans

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Texte et affichage

```
body {
  font-family: Arial, sans-serif;
  font-size: 16px;
  line-height: 1.6;
  color: #16324f;
  background-color: #f4f7fa;
}
h1 { font-size: 32px; text-align: center; }
.note { font-weight: bold; font-style: italic; }
.cache { display: none; }
.etiquette { display: inline-block; padding: 4.8px; }
```

color règle la couleur du texte ; background-color celle du fond. Une couleur peut être un nom, une valeur hexadécimale, rgb() ou hsl(). opacity affecte l'élément entier et ses descendants ; une couleur avec alpha permet de ne rendre transparent que le fond.

Bordures et fonds

```
.carte {
  border: 2px solid #167d8d;
  border-radius: 8px;
  padding: 16px;
  margin: 16px 0;
  background: white;
}
.banniere {
  background: #16324f url("fond.png") no-repeat center;
  background-size: cover;
}
.degrade { background: linear-gradient(90deg, teal, navy); }
```

border-top, border-right, border-bottom et border-left ciblent un côté. border-width, border-style et border-color règlent séparément épaisseur, style et couleur. Exemples de styles : solid, dashed, dotted, double, inset.

background-repeat prend notamment repeat, no-repeat, repeat-x ou repeat-y ; background-attachment: fixed fixe l'image au viewport dans les situations usuelles ; background-position règle sa position. radial-gradient() crée un dégradé radial.

Application : modèle de boîte

Créer un div de largeur 400px, avec un padding supérieur de 40px, une bordure verte de 10px et une marge de 30px. Comparer sa largeur extérieure avec et sans border-box.

Vérification

Utiliser l'onglet « Éléments » du navigateur pour inspecter les styles calculés et le schéma du modèle de boîte.

Positionnement : relative et absolute

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Flux normal et propriété position

Dans le flux normal, les éléments occupent leur place selon leur type d'affichage. position: static est la valeur initiale. top, right, bottom et left servent à décaler des éléments positionnés.

Position relative

```
.etiquette {
  position: relative;
  top: 5px;
  left: 10px;
}
```

L'élément est décalé visuellement, mais sa place initiale reste réservée dans le flux. Il peut recouvrir un voisin sans que celui-ci se déplace.

Position absolute

```
<div class="carte">
  <img src="fleur.jpg" alt="Bouquet de fleurs">
  <span class="badge">Nouveau</span>
</div>
```

```
.carte { position: relative; width: 300px; }
.carte img { display: block; width: 100%; }
.badge {
  position: absolute;
  top: 10px;
  right: 10px;
  padding: 6.4px;
  color: white;
  background: #147c89;
}
```

Un élément absolute est retiré du flux. Dans le cas courant illustré ici, il se place par rapport au plus proche ancêtre positionné, la carte. Sans cet ancêtre, sa référence peut être le bloc conteneur initial. Certaines propriétés comme transform peuvent aussi créer un bloc de référence.

Empilement

z-index permet de régler l'ordre d'empilement dans les contextes où il s'applique. Une valeur très élevée ne permet pas toujours de dépasser un élément appartenant à un autre contexte d'empilement.

Application

Afficher un prix dans le coin inférieur droit d'une image. Retirer position: relative de la carte et expliquer le changement. Éviter de positionner tous les blocs en absolute pour construire une page entière.

Positionnement : fixed et sticky

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

fixed : position généralement liée à la fenêtre

```
.aide {
  position: fixed;
  bottom: 16px;
  right: 16px;
  padding: 11.2px;
  background: #16324f;
  color: white;
  z-index: 10;
}
```

L'élément est retiré du flux et reste généralement visible au défilement, relativement au viewport. Un ancêtre transformé peut modifier cette référence. Prévoir un espace suffisant pour ne pas cacher du contenu, notamment sur mobile.

sticky : ancrage pendant le défilement

```
.entete-table {
  position: sticky;
  top: 0;
  background: white;
  z-index: 2;
}
```

L'élément reste dans le flux. Il se comporte comme un élément normal jusqu'à atteindre le seuil fixé par top, puis reste ancré dans les limites de son contexte de défilement et de son conteneur. Sans seuil pertinent, l'effet attendu peut ne pas apparaître.

Comparaison

- static : flux normal, sans décalage par les propriétés de position.
- relative : place conservée, décalage visuel possible.
- absolute : hors flux, référence à un bloc conteneur.
- fixed : hors flux, référence généralement au viewport.
- sticky : place conservée, ancrage conditionné par le défilement.

Points de contrôle

Un ancêtre ayant overflow: auto ou hidden peut modifier le comportement de sticky. Une zone trop courte ne laisse pas assez d'espace pour observer l'ancrage. Vérifier aussi que le contenu reste accessible au clavier et à l'agrandissement du texte.

Application

Créer un en-tête sticky et un bouton d'aide fixed. Tester le défilement, le zoom et une fenêtre étroite. Vérifier qu'aucun champ de formulaire n'est caché par l'interface fixe.

Mise en page avec float et clear

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Flottants

float: left place un élément à gauche et permet au contenu suivant de l’entourer. float: right le place à droite. clear évite qu’un élément reste à côté de flottants antérieurs.

```
<header><img class="logo" src="logo.png" alt="Logo du lycée">
  <h1>Gestion des élèves</h1></header>
<div class="milieu">
  <nav><a href="inscription.html" target="cadre">Inscription</a></nav>
  <section class="contenu">
    <iframe name="cadre" title="Travail"
            src="inscription.html"></iframe>
  </section>
</div>
<footer>Lycée Rafèha</footer>
```

```
* { box-sizing: border-box; }
.logo { float: left; width: 70px; margin-right: 16px; }
header { display: flow-root; }
nav { float: left; width: 180px; }
nav a { display: block; padding: 9.6px; }
.contenu { margin-left: 200px; }
.milieu { display: flow-root; }
iframe { width: 100%; min-height: 500px; border: 0; }
footer { clear: both; text-align: center; }
```

Contenir les flottants

Un conteneur dont tous les enfants flottent peut ne pas les inclure dans sa hauteur ordinaire. display: flow-root crée un contexte de formatage qui contient les flottants. clear: both peut faire commencer le pied de page sous les flottants précédents.

Contraintes de l’activité

Les projets « Gestion des élèves » et « Recharge électrique » de la plateforme travaillent explicitement float et clear. Dans ces exercices, respecter cette consigne plutôt que remplacer immédiatement la mise en page par flex.

Application

Construire une navigation de largeur fixe et une zone de contenu à droite. Ajouter une transition aux liens. Tester les débordements lorsque la fenêtre devient étroite ; proposer ensuite une règle adaptée aux petits écrans.

Flexbox, affichage et débordement

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Affichage

block occupe généralement une ligne disponible ; inline s’intègre au texte ; inline-block combine un placement en ligne et une boîte dimensionnable. display: none retire l’élément de l’affichage et de la mise en page ; visibility: hidden le masque en conservant sa place.

Conteneur flexible

```
<div class="cartes">
  <article>HTML5</article>
  <article>CSS3</article>
  <article>JavaScript</article>
</div>
```

```
.cartes {
  display: flex;
  flex-direction: row;
  flex-wrap: wrap;
  gap: 16px;
  justify-content: space-between;
  align-items: stretch;
}
.cartes article {
  flex: 1 1 180px;
  padding: 16px;
  border: 1px solid teal;
}
```

justify-content agit sur l’axe principal ; align-items sur l’axe transversal. Changer flex-direction change ces axes. gap ajoute un espace entre les éléments. flex-wrap autorise le retour à la ligne.

Débordement

```
.zone {
  max-height: 200px;
  overflow: auto;
}
.texte-long { overflow-wrap: anywhere; }
```

overflow: visible laisse déborder ; hidden masque le débordement ; auto ajoute un mécanisme de défilement si nécessaire ; scroll le prévoit même si le contenu ne déborde pas dans les situations usuelles. Vérifier que masquer un débordement ne rend pas une information inaccessible.

Application

Afficher trois cartes flexibles qui passent sur plusieurs lignes. Construire ensuite le même écran avec float pour comparer les mécanismes. L’emploi de Flexbox est un complément ; il ne remplace pas la consigne float d’un TP qui l’impose.

Liens, listes, tableaux et ombres

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

États des liens et focus

```
a:link { color: navy; }
a:visited { color: purple; }
a:hover { color: darkred; }
a:active { color: orange; }
a:focus-visible { outline: 3px solid teal; }
```

L'ordre link, visited, hover, active aide à éviter des conflits courants à spécificité égale. Le navigateur restreint les styles de visited pour protéger la confidentialité. Ne pas supprimer l'indicateur de focus sans solution de remplacement.

Listes

```
ul { list-style-type: square; list-style-position: outside; }
.menu { list-style: none; padding-left: 0; }
.illustrations { list-style-image: url("puce.png"); }
```

list-style combine type, position et image. Lorsqu'une image de puce valide est disponible, elle est utilisée à la place du type de puce.

Tableaux

```
table { width: 100%; border-collapse: collapse; }
th, td { border: 1px solid #9aabb8; padding: 8px; }
th { background: #e8f3f5; text-align: left; }
tr:nth-child(even) { background: #f4f7fa; }
```

border-collapse: collapse fusionne les bordures voisines. table-layout: fixed fixe davantage le calcul des largeurs à partir de la table et de ses colonnes ; gérer les contenus longs.

Ombres et arrondis

```
.carte {
  border-radius: 12px 0 12px 0;
  box-shadow: 0 4px 12px rgb(0 0 0 / 20%);
}
h1 { text-shadow: 1px 2px 3px #b7c4cc; }
```

box-shadow : décalage horizontal, vertical, flou, puis éventuellement étalement et couleur. text-shadow s'applique au texte. Des valeurs multiples séparées par des virgules produisent plusieurs ombres. Conserver un contraste et une lisibilité suffisants.

Filtres et transformations 2D

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Filtres étudiés

```
.flou { filter: blur(3px); }
.gris { filter: grayscale(100%); }
.inverse { filter: invert(100%); }
.effet { filter: grayscale(100%) blur(1px); }
```

blur utilise une longueur positive ou nulle ; grayscale et invert admettent notamment un pourcentage de 0 % à 100 %. Leur valeur neutre est 0 ou 0 %. L'ordre des fonctions peut modifier le résultat. Le filtre s'applique au rendu de l'élément, y compris son contenu ; il ne modifie pas le fichier source.

Transformations

```
.rotation { transform: rotate(15deg); }
.taille { transform: scale(1.1); }
.decalage { transform: translate(20px, 10px); }
.inclinaison { transform: skew(10deg, 0deg); }
.combine {
  transform-origin: center;
  transform: translateX(15px) rotate(10deg) scale(1.05);
}
```

rotate tourne ; scale change l'échelle ; translate déplace ; skew incline. transform-origin fixe le point d'origine des transformations. Les fonctions se composent : modifier leur ordre peut changer l'effet observé.

Transform et mise en page

Une transformation visuelle ne redistribue pas le flux comme une modification des dimensions ordinaires. L'élément peut donc chevaucher un voisin. Un élément transformé peut aussi créer un nouveau contexte d'empilement et un bloc de référence pour certains descendants positionnés.

Survol progressif

```
.photo { transition: transform 0.3s, filter 0.3s; }
.photo:hover {
  transform: scale(1.05);
  filter: grayscale(100%);
}
```

Application

Créer une carte de fleur qui grandit légèrement au survol, puis tester blur, grayscale et invert séparément. Ajouter la même mise en évidence au focus si la carte est interactive ; ne pas réserver une information indispensable au seul survol.

Transitions et animations

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Transition entre deux états

Syntaxe simplifiée : propriété durée fonction-de-temporisation délai.

```
nav a {
  display: block;
  background-color: #edf5f6;
  transition: background-color 0.3s ease,
              transform 0.3s ease-out;
}
nav a:hover, nav a:focus-visible {
  background-color: #bfe5e9;
  transform: translateX(5px);
}
```

linear garde une progression régulière ; ease accélère puis ralentit ; ease-in privilégie un départ lent ; ease-out une fin lente ; ease-in-out combine les deux. Éviter transition: all si seules quelques propriétés doivent changer.

Animation avec étapes

```
@keyframes apparition {
  from { opacity: 0; transform: translateY(15px); }
  to { opacity: 1; transform: translateY(0); }
}
.carte {
  animation: apparition 0.6s ease-out 0s 1 normal both;
}
```

Les valeurs abrégées indiquent ici nom, durée, temporisation, délai, nombre d’itérations, sens et mode de remplissage. Des étapes 0 %, 50 % et 100 % permettent plusieurs états. animation-iteration-count: infinite répète sans fin ; éviter les mouvements permanents inutiles.

Réduire les mouvements

```
@media (prefers-reduced-motion: reduce) {
  .carte { animation: none; }
  nav a { transition: none; }
}
```

Application

Faire apparaître une carte une seule fois. Ajouter une transition de bordure aux champs ayant le focus. Vérifier le comportement lorsque l’utilisateur préfère moins d’animations. Les effets visuels ne doivent pas retarder la lecture ou masquer les erreurs de formulaire.

Intégration, variables et types

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Rôle et chargement

Dans ce cours, JavaScript s’exécute dans le navigateur : il réagit aux événements, manipule le DOM et peut échanger avec un serveur. Il existe aussi des environnements JavaScript côté serveur.

```
<script src="script.js" defer></script>
<!-- Ou script interne placé à la fin de body -->
<script>
  console.log("Page prête");
</script>
```

Déclarer avec let

```
let nombre = 2;
let texte = "Bonjour";
let actif = true;
let resultat;                 // undefined
let selection = null;
nombre = nombre + 3;
console.log(nombre);          // 5
```

let déclare une variable dont la portée est le bloc délimité par des accolades. Il faut la déclarer avant de l’utiliser et ne pas la redéclarer dans le même bloc. Le cours utilise let pour les variables ; const reste possible pour une liaison qui ne sera pas réaffectée.

```
let message = "À l'extérieur";
if (actif) {
  let message = "Dans le bloc";
  console.log(message);
}
console.log(message);         // À l'extérieur
```

Types et identifiants

Les types usuels comprennent number, string, boolean, undefined, object et bigint ; null représente explicitement une absence de valeur. Les tableaux sont des objets. typeof null vaut "object", particularité historique du langage.

Un identifiant est sensible à la casse, ne commence pas par un chiffre et ne peut pas être un mot réservé. Préférer des noms explicites : note, total, nomEleve.

Commentaires et instructions

// commence un commentaire sur une ligne ; /* ... */ un commentaire de plusieurs lignes. Terminer les instructions par un point-virgule dans les exemples du cours.

Application

Déclarer le nom d’un élève, sa moyenne et son état d’admission, puis afficher leurs types avec typeof.

Opérateurs, conversions et dialogues

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Calcul et concaténation

```
let a = 7, b = 2;
console.log(a + b, a - b, a * b); // 9, 5, 14
console.log(a / b, a % b, a ** b); // 3.5, 1, 49
let prenom = "Amal";
let message = "Bonjour " + prenom;
console.log(message);
console.log(4 + "2");            // "42"
console.log(Number("4") + 2);     // 6
```

+=, -=, *= et /= combinent une opération et une affectation. ++ ajoute une unité ; -- soustrait une unité.

Dialogues du navigateur

alert(message) affiche un message. prompt(message) renvoie une chaîne ou null en cas d'annulation. confirm(message) renvoie true pour OK et false pour Annuler.

```
let saisie = prompt("Saisir une note de 0 à 20 :");
if (saisie !== null && saisie.trim() !== "") {
  let note = Number(saisie);
  if (!isNaN(note) && note >= 0 && note <= 20) {
    alert("Note : " + note);
  } else {
    alert("Note invalide.");
  }
}
```

Convertir correctement

• Number("12.5") produit 12.5 ; Number("12abc") produit NaN.

• Number("") produit 0 : vérifier une saisie vide avant conversion.

• Number.parseInt("12", 10) analyse un entier en base 10.

• Number.parseFloat("12.5abc") produit 12.5 : cette fonction ne valide pas toute la chaîne.

• String(42) produit "42" ; Number.isNaN(x) teste la valeur NaN sans conversion.

La base 10 explicite de parseInt rend l'intention claire. Sans base, l'analyse moderne utilise généralement 10, sauf notamment un préfixe 0x interprété en base 16.

Application

Demander deux nombres, gérer l'annulation et les chaînes vides, puis afficher leur somme après validation.

Conditions et choix multiples

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Comparaisons et logique

=== compare la valeur et le type ; !== teste leur différence. == et != peuvent convertir les opérandes : privilégier les comparaisons strictes. Les autres comparaisons sont <, <=, > et >=.

&& exprime ET ; || exprime OU ; ! exprime NON. Ces opérateurs utilisent des évaluations booléennes, mais && et || peuvent renvoyer un opérande non booléen.

```
let note = 14;
let resultat;
if (note < 0 || note > 20) {
  resultat = "Note invalide";
} else if (note >= 10) {
  resultat = "Admis";
} else {
  resultat = "Non admis";
}
console.log(resultat);
```

La variable resultat est déclarée avant les blocs, car elle est utilisée ensuite. Une variable déclarée avec let à l'intérieur d'un seul bloc ne serait pas accessible à cet endroit.

switch

```
let fruit = "Pommes";
switch (fruit) {
  case "Oranges":
    console.log("2 DT le kilo");
    break;
  case "Pommes":
    console.log("3,5 DT le kilo");
    break;
  case "Bananes":
  case "Cerises":
    console.log("9,5 DT le kilo");
    break;
  default:
    console.log("Fruit non disponible");
}
```

switch compare strictement l'expression aux valeurs des case. break évite de poursuivre dans le cas suivant.

Opérateur conditionnel

let etat = note >= 10 ? "Admis" : "Non admis";

Cette expression choisit l'une des deux valeurs ; la réserver à des choix simples pour garder un code lisible.

SCANNER LA FICHE
ou cliquer sur le QR code

Boucles et accumulation

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

for : nombre d’itérations connu

```
let somme = 0;
for (let i = 1; i <= 10; i++) {
  somme += i;
}
console.log(somme); // 55 ; i n'est pas accessible ici.
```

La condition du for est une condition de poursuite : le bloc s’exécute tant qu’elle est vraie.

while : test avant l’exécution

```
let nombre = 1;
while (nombre < 10) {
  nombre++;
}
console.log(nombre); // 10
```

do...while : au moins une exécution

```
let total = 0;
let continuer = true;
do {
  let saisie = prompt("Nombre positif ; négatif pour finir :");
  if (saisie === null) { break; }
  let nombre = Number(saisie);
  if (saisie.trim() === "" || (isNaN(nombre) || nombre === Infinity || nombre === -Infinity)) {
    alert("Saisie invalide");
    continue;
  }
  continuer = nombre >= 0;
  if (continuer) { total += nombre; }
} while (continuer);
alert("Somme : " + total);
```

continue passe à l’itération suivante ; break quitte la boucle. continuer est initialisé à true : une saisie invalide fait recommencer la boucle sans modifier le total. Le nombre négatif de fin n’est pas ajouté à la somme.

Compter des prénoms

```
let prenoms = [], prenom;
while ((prenom = prompt("Prénom ; vide pour terminer :")) !== null) {
  if (prenom.trim() === "") { break; }
  prenoms[prenoms.length] = prenom.trim();
}
let listePrenoms = "";
for (let i = 0; i < prenoms.length; i++) {
  if (i > 0) { listePrenoms += ", "; }
  listePrenoms += prenoms[i];
}
alert(prenoms.length + " prénoms : " + listePrenoms);
```

Vigilance

Prévoir une progression et une condition d’arrêt. Une boucle synchrone infinie bloque l’interface du navigateur.

TP : jeu de devinettes

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Objectif

Générer un entier entre 1 et 12 puis permettre à l'utilisateur de le deviner. Compter seulement les propositions valides et autoriser l'annulation.

```
let minimum = 1;
let maximum = 12;
let secret = Math.floor(Math.random() *
             (maximum - minimum + 1)) + minimum;
let essais = 0;
let trouve = false;
let indice = "";
while (!trouve) {
  let saisie = prompt("Entier de 1 à 12. " + indice);
  if (saisie === null) {
    alert("Partie annulée.");
    break;
  }
  let nombre = Number(saisie);
  if (saisie.trim() === "" || !Number.isInteger(nombre)
      || nombre < minimum || nombre > maximum) {
    indice = "La proposition doit être un entier valide.";
    continue;
  }
  essais++;
  if (nombre > secret) {
    indice = "Cherchez plus petit.";
  } else if (nombre < secret) {
    indice = "Cherchez plus grand.";
  } else {
    trouve = true;
    alert("Bravo ! Trouvé en " + essais + " essai(s).");
  }
}
```

Comprendre le tirage

Math.random() renvoie un réel dans [0, 1[. Math.floor(x) renvoie le plus grand entier inférieur ou égal à x. Le calcul multiplie par le nombre de valeurs possibles, puis ajoute la borne minimale.

Questions

1. Pourquoi ne faut-il pas utiliser Math.round() dans cette formule ?

2. À quel endroit la portée de la variable nombre se termine-t-elle ?

3. Que devient essais lorsque l'utilisateur saisit une lettre ?

Améliorations

Limiter le nombre d'essais, proposer une nouvelle partie et remplacer prompt par un formulaire HTML. Math.random() ne convient pas à la génération de secrets de sécurité.

DOM : sélectionner et modifier des éléments

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Accéder aux éléments du document

Le DOM représente la page sous forme d’objets. getElementById renvoie un élément ou null. getElementsByTagName, getElementsByClassName et getElementsByName renvoient des collections, à parcourir par indices.

```
<p id="intro" class="carte">Un texte initial.</p>
<img id="photo" src="avant.png" alt="Version initiale">
```

```
let paragraphe = document.getElementById("intro");
if (paragraphe !== null) {
  paragraphe.textContent = "Un autre texte";
  paragraphe.classList.add("important");
}
let photo = document.getElementById("photo");
if (photo !== null) {
  photo.setAttribute("src", "apres.png");
  photo.alt = "Nouvelle version";
}
```

Parcourir une collection

```
let cartes = document.getElementsByClassName("carte");
for (let i = 0; i < cartes.length; i++) {
  cartes[i].classList.add("visible");
}
let paragraphes = document.getElementsByTagName("p");
console.log(paragraphes.length);
```

Une collection DOM n’est pas nécessairement un tableau Array. Ces collections peuvent être dynamiques : modifier les classes d’appartenance pendant leur parcours peut changer leur contenu. L’exemple conserve la classe carte.

Autres accès et affichage

Par exemple, document.forms["inscription"] désigne un formulaire nommé inscription et document.images[0] la première image si elle existe. Les noms de propriétés sont sensibles à la casse. Charger le script avec defer après avoir défini les éléments HTML.

textContent affiche du texte sans l’interpréter comme HTML. innerHTML interprète du balisage : ne pas lui affecter une saisie libre non protégée.

DOM : lire et contrôler un formulaire

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Propriétés essentielles

value contient la valeur textuelle d'un champ. checked indique l'état d'une case ou d'un bouton radio. Pour une liste, options est la collection des choix ; selectedIndex est l'indice sélectionné, ou -1. Chaque option possède text, value et selected.

```
<form id="calcul" onsubmit="return validerNote()">
  <label for="note">Note</label>
  <input id="note" name="note" type="number"
         min="0" max="20" step="0.5" required>
  <label><input id="accord" type="checkbox"> Je confirme</label>
  <button type="submit">Vérifier</button>
</form>
<p id="resultat"></p>
```

Une fonction de contrôle qui renvoie un booléen

```
function validerNote() {
  let champ = document.getElementById("note");
  let accord = document.getElementById("accord").checked;
  let note = Number(champ.value);
  let sortie = document.getElementById("resultat");
  if (champ.value.trim() === "" || isNaN(note)
      || note < 0 || note > 20 || !accord) {
    sortie.textContent = "Vérifier les données.";
    return false;
  }
  sortie.textContent = note >= 10 ? "Admis" : "Non admis";
  return false; // Démonstration locale : aucun envoi au serveur.
}
```

Dans onsubmit, return transmet le résultat : false empêche l'envoi. Dans un vrai formulaire de traitement, renvoyer true quand toutes les données sont valides. Les bornes 0 et 20 excluent aussi les valeurs infinies.

Lire une liste ou un groupe radio existant

```
let liste = document.getElementById("classe");
if (liste !== null && liste.selectedIndex >= 0) {
  console.log(liste.options[liste.selectedIndex].value);
}
let radios = document.getElementsByName("groupe");
for (let i = 0; i < radios.length; i++) {
  if (radios[i].checked) { console.log(radios[i].value); }
}
```

Les choix sont définis directement par les balises option du HTML. Vérifier à nouveau les données côté serveur.

Fonctions, portée et événements

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Définir et appeler une fonction

```
function doubler(nombre) {
  let resultat = nombre * 2;
  return resultat;
}
let valeur = doubler(7);
console.log(valeur); // 14
```

Un paramètre est un nom dans la définition ; un argument est une valeur transmise lors de l'appel. return termine la fonction et renvoie une valeur. Sans retour explicite, le résultat est undefined.

Portée

```
let message = "Message extérieur";
function afficher() {
  let message = "Message local";
  console.log(message);
}
afficher();
console.log(message); // Message extérieur
```

Les arguments sont transmis par valeur. Pour un objet, la valeur transmise est une référence : modifier ses propriétés peut modifier l'objet partagé, mais réaffecter le paramètre ne réaffecte pas la variable de l'appelant.

Associer un événement

```
<button id="bouton" type="button">Cliquer</button>
<p id="sortie"></p>
```

```
function reagir(event) {
  document.getElementById("sortie").textContent = "Bonjour";
  console.log(event.currentTarget.id);
}
let bouton = document.getElementById("bouton");
bouton.addEventListener("click", reagir);
// Pour retirer cet écouteur :
// bouton.removeEventListener("click", reagir);
```

Méthodes et événements usuels

onclick = reagir associe un gestionnaire, remplacé par une affectation ultérieure. addEventListener autorise plusieurs écouteurs et est recommandé. Ne pas écrire reagir() lors de l'association : cela appellerait immédiatement la fonction.

Événements courants : click, input, change, submit, focus et blur. event.target désigne la cible initiale ; event.currentTarget l'élément portant l'écouteur. Éviter les chaînes de code dans les attributs d'événements.

Nombres, chaînes et méthodes usuelles

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Nombres et Math

```
let nombre = Number("123");
console.log(nombre.toString());   // "123"
console.log((6).toString(2));      // "110"
console.log(Number.isNaN(NaN));    // true
console.log(Math.floor(4.8));      // 4
console.log(Math.round(4.8));      // 5
console.log(Math.abs(-7));        // 7
```

Number utilise généralement une représentation IEEE 754 en double précision. Certains décimaux ne sont pas représentés exactement. Éviter new Number() pour un simple nombre : il crée un objet dont valueOf() extrait la valeur primitive.

Chaînes

```
let texte = "  Bonjour Tunis  ";
let propre = texte.trim();
console.log(propre.length);              // 13
console.log(propre[0]);           // "B"
console.log(propre.indexOf("Tunis"));    // 8
console.log(propre.lastIndexOf("o"));   // 4
console.log(propre.substring(0, 7));     // "Bonjour"
console.log(propre.slice(-5));           // "Tunis"
console.log(propre.toUpperCase());       // "BONJOUR TUNIS"
console.log(propre.toLowerCase());       // "bonjour tunis"
console.log("Bonjour ".concat("Amal")); // "Bonjour Amal"
```

indexOf et lastIndexOf renvoient -1 en l'absence de correspondance. substring utilise un indice de fin exclu et échange les bornes si nécessaire. slice accepte des indices négatifs. L'ancienne méthode substr est à éviter dans de nouveaux exemples.

length mesure des unités de code UTF-16, pas toujours des caractères visibles : certains symboles comptent pour deux unités ou davantage. Les chaînes sont immuables : les méthodes renvoient de nouvelles valeurs. Lire chaine[indice] donne l'unité de code correspondante ; un indice hors limites renvoie undefined.

Pourquoi éviter eval ?

eval exécute une chaîne comme du code. Il ne faut pas l'utiliser pour calculer une expression saisie par l'utilisateur. Employer les opérateurs et les conversions explicites.

Application

Nettoyer un nom avec trim, le convertir en majuscules et vérifier qu'il contient au moins deux caractères.

Tableaux et parcours

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Créer, lire et ajouter par indice

```
let fruits = ["Pomme", "Banane"];
console.log(fruits[0]); // Pomme
fruits[fruits.length] = "Orange";
console.log(fruits.length); // 3
```

Pour un tableau dense, les indices vont de 0 à length − 1. Un indice absent renvoie undefined. Une affectation très éloignée peut créer des cases absentes : un simple count des éléments présents n'est alors pas la longueur.

Insérer et supprimer avec des décalages

```
for (let i = fruits.length; i > 0; i--) {
  fruits[i] = fruits[i - 1];
}
fruits[0] = "Fraise"; // insertion au début
let dernier = fruits[fruits.length - 1];
fruits.length = fruits.length - 1; // retrait du dernier élément
for (let i = 0; i < fruits.length - 1; i++) {
  fruits[i] = fruits[i + 1];
}
fruits.length = fruits.length - 1; // retrait du premier élément
```

Ces deux retraits supposent un tableau non vide, comme ici. En général, tester length > 0 avant de réduire sa longueur. Le décalage modifie les indices des éléments.

Parcourir et construire un affichage

```
let message = "";
for (let i = 0; i < fruits.length; i++) {
  if (i > 0) { message += ", "; }
  message += fruits[i];
}
console.log(message); // Pomme, Banane
```

Moyenne et tri numérique

```
let notes = [12, 15, 9, 16];
let somme = 0;
for (let i = 0; i < notes.length; i++) { somme += notes[i]; }
let moyenne = notes.length > 0 ? somme / notes.length : null;
console.log(moyenne); // 13
notes.sort(function (a, b) { return a - b; });
console.log(notes); // [9, 12, 15, 16]
```

Sans fonction de comparaison, sort compare les représentations textuelles. Fournir a − b pour des nombres croissants. Prévoir un tableau vide avant de calculer une moyenne.

SCANNER LA FICHE
ou cliquer sur le QR code

Dates, temporisation et horloge numérique

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Objet Date

```
let maintenant = new Date();
let exemple = new Date(2026, 9, 21, 12, 30, 0);
console.log(exemple.getFullYear()); // 2026
console.log(exemple.getMonth());    // 9 : octobre
console.log(exemple.getDate());     // 21 : jour du mois
```

getDay renvoie le jour de la semaine de 0 (dimanche) à 6. getHours, getMinutes, getSeconds et getMilliseconds renvoient respectivement 0–23, 0–59, 0–59 et 0–999. Ces méthodes lisent l'heure locale du navigateur.

getTime() renvoie les millisecondes depuis le 1er janvier 1970 à 00:00:00 UTC ; setTime(ms) modifie la date. En PHP, un timestamp Unix est exprimé en secondes : ne pas confondre les unités.

TP : horloge

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Horloge</title>
</head>
<body>
  <span id="horloge" aria-label="Heure actuelle"></span>
  <script>
    function afficherHorloge() {
      let date = new Date();
      let h = (date.getHours() < 10 ? "0" : "") + date.getHours();
      let m = (date.getMinutes() < 10 ? "0" : "") + date.getMinutes();
      let s = (date.getSeconds() < 10 ? "0" : "") + date.getSeconds();
      document.getElementById("horloge").textContent =
        h + ":" + m + ":" + s;
    }
    afficherHorloge();
    let minuterie = setInterval(afficherHorloge, 1000);
    // clearInterval(minuterie) permet l'arrêt.
  </script>
</body>
</html>
```

setTimeout(fonction, délai) prévoit un appel unique ; setInterval(fonction, délai) prévoit des appels répétés. Le délai est en millisecondes et n'est pas une garantie d'exécution exacte. Passer une fonction plutôt qu'une chaîne de code.

Compléments : Math, chaînes et sorties

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Fonctions Math et valeurs numériques

```
console.log(Math.ceil(4.2));       // 5
console.log(Math.trunc(-4.8));     // -4
console.log(Math.sqrt(81));       // 9
console.log(Math.pow(2, 3));       // 8
console.log(Math.min(5, 2, 9));    // 2
console.log(Math.max(5, 2, 9));    // 9
console.log(Math.round(12.345 * 100) / 100); // 12.35
console.log(isNaN("abc"));        // true : conversion préalable
console.log(Number.isNaN("abc")); // false : pas de conversion
```

L'arrondi ci-dessus renvoie un nombre et ne garantit pas deux chiffres après la virgule à l'affichage. Les décimaux restent sujets à leur représentation binaire. Math.floor(-4.8) vaut -5, tandis que Math.trunc(-4.8) vaut -4.

Rechercher une sous-chaîne avec indexOf

```
let texte = "HTML et HTML";
let position = texte.indexOf("HTML");
console.log(position); // 0
if (position !== -1) { console.log("Texte trouvé"); }
if (texte.indexOf("HTML") === 0) { console.log("Au début"); }
```

indexOf renvoie -1 si la sous-chaîne est absente. Le résultat 0 est une position valide. Les comparaisons de chaînes sont sensibles à la casse.

Sorties et concaténation

console.log aide au débogage ; alert interrompt l'interaction. textContent affiche du texte ; innerHTML interprète du HTML. document.write peut remplacer le document après son chargement : éviter son emploi dans de nouveaux exemples.

```
console.log(1 + 2 + "3"); // "33"
console.log("1" + 2 + 3); // "123"
console.log("Total : " + (2 + 3)); // Total : 5
```

Contre-exemples barrés : hors des exemples actifs

```
// ~~Number.isFinite(nombre)~~; // Non retenu dans ce cours.
// new ~~Option("Texte", "valeur")~~; // Non retenu dans ce cours.
```

Pour les contrôles du cours, convertir avec Number puis vérifier isNaN et les bornes du domaine. Les options de listes sont écrites dans le HTML.

Médias, styles et dates de formulaire

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Contrôler un média existant

```
<video id="film" controls src="demo.mp4"></video>
<button id="lecture" type="button">Lire</button>
<button id="pause" type="button">Pause</button>
<p id="message"></p>
```

```
let film = document.getElementById("film");
document.getElementById("lecture").addEventListener("click",
  function () {
    film.play().catch(function () {
      document.getElementById("message").textContent = "Lecture impossible.";
    });
  });
document.getElementById("pause").addEventListener("click",
  function () { film.pause(); });
```

Le navigateur peut refuser la lecture ; un clic utilisateur est préférable à un démarrage automatique. La gestion du rejet évite de laisser l'échec sans message.

Modifier une présentation explicitement

```
let element = document.getElementById("message");
element.style.backgroundColor = "#edf5f6";
let important = true;
element.className = important ? "important" : "";
```

className remplace toute la chaîne de classes : conserver explicitement celles qui sont nécessaires. Les propriétés CSS avec tirets deviennent généralement camelCase en JavaScript.

Contrôler une date civile non future

```
function dateNonFuture(valeur) {
  if (valeur.length !== 10 || valeur[4] !== "-"
      || valeur[7] !== "-") { return false; }
  for (let i = 0; i < valeur.length; i++) {
    if (i !== 4 && i !== 7 && (valeur[i] < "0" || valeur[i] > "9")) {
      return false;
    }
  }
  let a = Number(valeur.substring(0, 4));
  let m = Number(valeur.substring(5, 7));
  let j = Number(valeur.substring(8, 10));
  if (a < 1 || m < 1 || m > 12 || j < 1 || j > 31) { return false; }
  let date = new Date(0);
  date.setFullYear(a, m - 1, j);
  date.setHours(0, 0, 0, 0);
  let limite = new Date();
  limite.setHours(0, 0, 0, 0);
  return date.getFullYear() === a && date.getMonth() === m - 1
    && date.getDate() === j && date <= limite;
}
```

Lire année, mois et jour par indices et sous-chaînes, puis comparer les composantes reconstruites : le 31 février doit être rejeté. Le serveur doit refaire ces contrôles avec son propre fuseau.

SCANNER LA FICHE
ou cliquer sur le QR code

Notions relationnelles et contraintes

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Vocabulaire relationnel

Une base de données (BD) est un ensemble organisé et persistant de données. Un SGBD sert à définir, manipuler et contrôler ces données ; MySQL est un SGBD relationnel et SQL est le langage utilisé pour l'interroger.

• Table : ensemble de lignes décrites par des colonnes.

• Champ ou colonne : propriété ayant un nom, un type et des contraintes.

• Ligne, enregistrement ou n-uplet : occurrence décrite par les champs.

• Clé primaire (PK) : champ, ou ensemble de champs, unique et non nul identifiant une ligne.

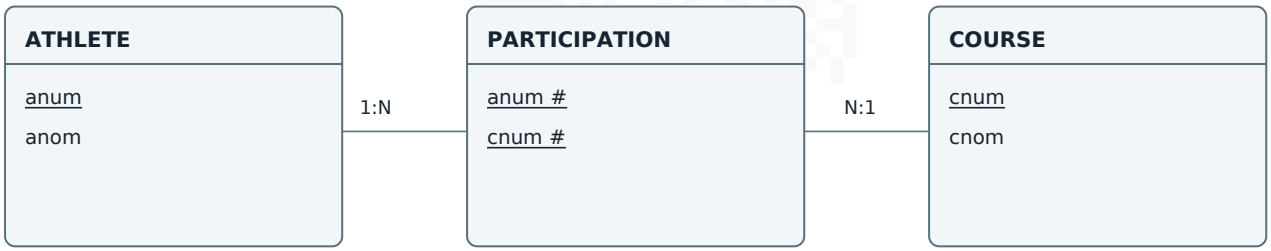
• Clé étrangère (FK) : champ(s) référençant une clé d'une table ; elle assure la cohérence des liens.

Représentation graphique des relations

UN À PLUSIEURS — CLASSE / ÉLÈVE



PLUSIEURS À PLUSIEURS — ASSOCIATION PARTICIPATION



PK composée : (anum, cnum)

Lire les schémas

Les clés primaires sont soulignées. # signale une clé étrangère dans ce schéma pédagogique, pas un caractère à ajouter dans un identifiant SQL. Une classe accueille zéro à plusieurs élèves ; chaque élève appartient ici à exactement une classe. L'association PARTICIPATION transforme la relation plusieurs-à-plusieurs en deux relations un-à-plusieurs. Ses deux champs forment ensemble une seule clé primaire composée.

Contraintes et valeurs absentes

NOT NULL impose une valeur ; UNIQUE impose l'unicité ; CHECK impose une condition ; FOREIGN KEY définit une référence. NULL signifie une information absente ou inconnue, pas une chaîne vide ni zéro. Les règles d'accès et de confidentialité doivent être définies : le simple stockage en base ne les garantit pas.

SQL : commandes et types MySQL

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Familles de commandes

• Définition des données : CREATE, ALTER, DROP.

• Manipulation des données : INSERT, UPDATE, DELETE ; interrogation : SELECT.

Les exemples suivants ciblent MySQL 8.0.16 ou ultérieur, avec le moteur InnoDB. À partir de cette version, les contraintes CHECK sont effectivement appliquées. Des différences peuvent exister avec MariaDB et d'autres SGBD.

```
CREATE DATABASE gestion_location
  CHARACTER SET utf8mb4;
USE gestion_location;
SHOW DATABASES;
SHOW TABLES;
```

Types numériques

• TINYINT : entier signé de -128 à 127 ; UNSIGNED de 0 à 255.

• INT : entier signé de -2147483648 à 2147483647 ; UNSIGNED de 0 à 4294967295.

• DECIMAL(p,s) : décimal exact, p chiffres au total dont s après la virgule. DECIMAL(8,3) autorise cinq chiffres avant et trois après le point.

• NUMERIC est un synonyme de DECIMAL dans MySQL.

• FLOAT et DOUBLE sont approximatifs : préférer DECIMAL pour les montants monétaires.

Types textuels et binaires

• CHAR(n) : chaîne de longueur fixe ; VARCHAR(n) : longueur variable maximale n, limitée aussi par l'encodage et la taille maximale d'une ligne.

• TEXT : texte ; BLOB : données binaires. Leur taille est limitée en octets, et non toujours à 65535 caractères.

• La sensibilité à la casse d'un texte dépend de sa collation ; TEXT ne signifie pas « ASCII ».

• Téléphone et CIN sont des chaînes : un zéro initial doit être conservé.

Types temporels

DATE : 2026-10-21 ; TIME : 12:30:00 ; DATETIME : 2026-10-21 12:30:00 ; YEAR : 2026.

TIMESTAMP applique des conversions liées au fuseau horaire de la session ; DATETIME stocke une date et une heure civiles sans cette conversion automatique.

Conventions

Utiliser des identifiants sans espaces et conserver exactement la même casse pour les noms de tables. Les dates littérales sont écrites 'AAAA-MM-JJ'. Les chaînes utilisent des apostrophes droites.

Création des tables : gestion des locations

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Schéma cohérent

Créer les tables parentes avant la table qui les référence. La clé de location inclut la date pour permettre plusieurs locations d'un même véhicule par un même client à des dates différentes.

```
CREATE TABLE client (
  ncin CHAR(8) PRIMARY KEY,
  nom VARCHAR(50) NOT NULL,
  prenom VARCHAR(50) NOT NULL,
  adresse VARCHAR(100),
  ville VARCHAR(50) DEFAULT 'Tunis'
) ENGINE=InnoDB;

CREATE TABLE vehicule (
  immat VARCHAR(20) PRIMARY KEY,
  marque VARCHAR(30) NOT NULL,
  modele VARCHAR(30) NOT NULL,
  annee_acq YEAR NOT NULL,
  CONSTRAINT ck_annee CHECK (annee_acq >= 1990)
) ENGINE=InnoDB;

CREATE TABLE location (
  immat VARCHAR(20) NOT NULL,
  ncin CHAR(8) NOT NULL,
  date_loc DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  duree_loc INT NOT NULL,
  cout_loc DECIMAL(8,3) NOT NULL,
  PRIMARY KEY (immat, ncin, date_loc),
  CONSTRAINT fk_location_vehicule FOREIGN KEY (immat)
    REFERENCES vehicule(immat) ON DELETE RESTRICT,
  CONSTRAINT fk_location_client FOREIGN KEY (ncin)
    REFERENCES client(ncin) ON DELETE RESTRICT,
  CONSTRAINT ck_duree CHECK (duree_loc > 0),
  CONSTRAINT ck_cout CHECK (cout_loc > 0)
) ENGINE=InnoDB;
```

Intégrité référentielle

RESTRICT refuse la suppression d'un parent encore référencé. CASCADE supprime automatiquement ses enfants : à choisir seulement si cela correspond à la règle de gestion. Pour conserver un historique de locations, RESTRICT est plus prudent.

Dates et contraintes

Ne pas écrire CHECK(date_loc <= NOW()) : MySQL n'autorise pas cette fonction non déterministe dans un CHECK. Vérifier cette règle dans le traitement applicatif ou un déclencheur adapté. DEFAULT CURRENT_TIMESTAMP n'est pas réservé au seul type TIMESTAMP ; il est aussi utilisable avec DATETIME.

Modifier et supprimer une structure

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Exemples indépendants à tester sur une copie

```
ALTER TABLE client ADD COLUMN telephone VARCHAR(8);
ALTER TABLE client MODIFY COLUMN telephone VARCHAR(20);
ALTER TABLE client ALTER COLUMN ville SET DEFAULT 'Tunis';
ALTER TABLE client DROP COLUMN adresse;
ALTER TABLE vehicule ADD CONSTRAINT ck_annee_max
  CHECK (annee_acq <= 2026);
ALTER TABLE vehicule DROP CHECK ck_annee_max;
DESCRIBE client;
SHOW CREATE TABLE location;
```

Un numéro de téléphone reste textuel. Lors d’un MODIFY COLUMN, préciser les propriétés à conserver, notamment NOT NULL et DEFAULT, car certaines peuvent être perdues si on les omet.

Clé primaire composée

Pour une table location ancienne dont la clé était seulement (immat, ncin), la correction est :

```
ALTER TABLE location
  DROP PRIMARY KEY,
  ADD PRIMARY KEY (immat, ncin, date_loc);
```

La virgule sépare les deux modifications. Dans le schéma de la fiche précédente, la clé est déjà correcte : il n’est pas nécessaire de la modifier.

Contraintes et renommage

```
ALTER TABLE location DROP FOREIGN KEY fk_location_client;
ALTER TABLE vehicule RENAME TO voiture;
```

Ne pas supprimer une clé primaire référencée sans analyser les clés étrangères et les index associés. DISABLE KEYS et ENABLE KEYS ne désactivent pas les contraintes de clé primaire ou de clé étrangère InnoDB ; ce ne sont pas des commandes de suspension de l’intégrité.

Suppression : prudence

```
-- À exécuter seulement sur une base de test sauvegardée.
DROP TABLE location;
DROP TABLE vehicule;
DROP TABLE client;
-- DROP DATABASE gestion_location;
```

Supprimer les tables enfants avant les parentes. Ces commandes supposent que vehicule n’a pas été renommée. DROP détruit une structure ; DELETE supprime des lignes en conservant la table.

Application

Ajouter une colonne email au client, puis vérifier la nouvelle structure. Ne pas exécuter toutes les commandes de cette fiche à la suite sur la base servant aux autres exercices.

SCANNER LA FICHE
ou cliquer sur le QR code

Insérer, modifier et supprimer des lignes

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

INSERT : préciser les colonnes

```
INSERT INTO client (ncin, nom, prenom, ville)
VALUES ('01234567', 'Tounsi', 'Wael', 'Tunis');
INSERT INTO vehicule (immat, marque, modele, annee_acq)
VALUES ('123TU4567', 'Renault', 'Clio', 2026);
INSERT INTO location (immat, ncin, date_loc, duree_loc, cout_loc)
VALUES ('123TU4567', '01234567', '2026-10-21 09:00:00',
        3, 180.000);
```

Les valeurs doivent respecter les types, NOT NULL, l'unicité, les CHECK et les clés étrangères. Insérer les parents avant les enfants. Une chaîne numérique comme la CIN garde ses apostrophes pour préserver sa forme.

UPDATE : attention à WHERE

```
UPDATE client
SET ville = 'Sfax', telephone = '20123456'
WHERE ncin = '01234567';

UPDATE location
SET cout_loc = ROUND(cout_loc * 1.18, 3)
WHERE date_loc >= '2026-10-01'
  AND date_loc < '2026-11-01';
```

Le premier exemple suppose la colonne telephone ajoutée dans la fiche précédente. Le second augmente de 18 % le coût des locations d'octobre 2026. Sans WHERE, UPDATE modifierait toutes les lignes.

DELETE

```
DELETE FROM location
WHERE immat = '123TU4567'
  AND ncin = '01234567'
  AND date_loc = '2026-10-21 09:00:00';
```

Sans WHERE, DELETE FROM location supprime toutes les lignes de location. La suppression d'un parent référencé est refusée avec RESTRICT, ou propagée avec CASCADE si cette action a été définie.

Méthode de travail

1. Écrire un SELECT avec la même condition pour vérifier les lignes visées.
2. Vérifier la sauvegarde et, si nécessaire, utiliser une transaction.
3. Exécuter la modification et contrôler le nombre de lignes affectées.

Exercice

Changer la durée d'une location précisément identifiée, puis supprimer uniquement cette location. Expliquer pourquoi une condition portant seulement sur la CIN peut viser plusieurs lignes.

Base commerciale : schéma des exercices

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Un second domaine, indépendant des locations

Les exemples suivants utilisent une base de gestion commerciale. Chaque ligne de ventes représente la vente d’un article ; plusieurs lignes peuvent concerner le même client le même jour.

```
CREATE DATABASE commerce CHARACTER SET utf8mb4;
USE commerce;
CREATE TABLE fournisseurs (
  num_frs INT PRIMARY KEY,
  nom_frs VARCHAR(50) NOT NULL,
  tel_frs VARCHAR(20)
) ENGINE=InnoDB;
CREATE TABLE clients (
  cod_cli CHAR(4) PRIMARY KEY,
  nom_cli VARCHAR(50) NOT NULL,
  prn_cli VARCHAR(50) NOT NULL,
  nat_cli VARCHAR(30)
) ENGINE=InnoDB;
CREATE TABLE articles (
  num_art INT PRIMARY KEY,
  des_art VARCHAR(80) NOT NULL,
  col_art VARCHAR(30),
  pv_art DECIMAL(10,3) NOT NULL CHECK (pv_art > 0),
  pa_art DECIMAL(10,3) NOT NULL CHECK (pa_art > 0),
  num_frs INT NOT NULL,
  FOREIGN KEY (num_frs) REFERENCES fournisseurs(num_frs)
) ENGINE=InnoDB;
CREATE TABLE ventes (
  id_vnt INT AUTO_INCREMENT PRIMARY KEY,
  cod_cli CHAR(4) NOT NULL,
  num_art INT NOT NULL,
  dat_vnt DATETIME NOT NULL,
  qte_vnt INT NOT NULL CHECK (qte_vnt > 0),
  prix_unitaire DECIMAL(10,3) NOT NULL CHECK (prix_unitaire > 0),
  FOREIGN KEY (cod_cli) REFERENCES clients(cod_cli),
  FOREIGN KEY (num_art) REFERENCES articles(num_art)
) ENGINE=InnoDB;
CREATE TABLE magasins (
  num_mag INT PRIMARY KEY,
  loc_mag VARCHAR(50) NOT NULL
) ENGINE=InnoDB;
```

prix_unitaire conserve le prix réellement appliqué au moment de la vente. Un total historique doit utiliser ce prix, et non le prix courant de articles. Ce schéma simplifié ne sépare pas encore l’en-tête d’une facture et ses lignes.

SELECT, filtres et expressions

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Projection, sélection et DISTINCT

```
SELECT nom_cli, prn_cli FROM clients;
SELECT * FROM clients WHERE nat_cli = 'Tunisienne';
SELECT DISTINCT nat_cli FROM clients;
SELECT num_art, des_art, pv_art - pa_art AS marge
FROM articles;
SELECT CONCAT(prn_cli, ' ', nom_cli) AS nom_complet
FROM clients;
```

Une projection choisit des colonnes ; une sélection filtre des lignes. DISTINCT élimine les doublons du résultat. En MySQL, utiliser CONCAT pour concaténer : || est généralement un OU logique, sauf mode SQL particulier.

Conditions

```
SELECT * FROM articles WHERE pv_art BETWEEN 10 AND 70;
SELECT * FROM magasins WHERE loc_mag IN ('Sfax', 'Gabès');
SELECT * FROM magasins WHERE loc_mag NOT IN ('Sfax', 'Gabès');
SELECT * FROM articles
WHERE des_art LIKE 'Ch%' AND pv_art > 70;
SELECT * FROM fournisseurs WHERE nom_frs LIKE 'Ka____';
SELECT * FROM clients WHERE nat_cli IS NULL;
```

BETWEEN inclut ses bornes. IN teste l’appartenance à une liste ; LIKE recherche un motif : % correspond à zéro ou plusieurs caractères, _ à un seul caractère. 'Ka____' commence par Ka et compte six caractères.

Les opérateurs usuels sont =, <>, !=, <, <=, >, >= ; !< et !> ne sont pas des opérateurs MySQL valides. Combiner les conditions avec AND, OR et NOT et ajouter des parenthèses pour clarifier l’intention.

Dates : intervalle semi-ouvert

```
SELECT * FROM ventes
WHERE dat_vnt >= '2026-01-01'
  AND dat_vnt < '2026-01-09';
```

Cette condition couvre du 1er au 8 janvier inclus, même si dat_vnt contient une heure. Une borne supérieure '2026-01-08' ne couvrirait que minuit de cette dernière journée.

NULL et casse

Ne pas écrire = NULL : utiliser IS NULL ou IS NOT NULL. Les comparaisons de texte et LIKE dépendent de la collation ; UPPER peut normaliser la casse, mais ne corrige pas des libellés incohérents.

Agrégats, tri et sous-requêtes

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Fonctions d’agrégation

COUNT(*) compte les lignes ; COUNT(colonne) compte les valeurs non nulles ; COUNT(DISTINCT colonne) compte les valeurs distinctes non nulles. SUM, AVG, MIN et MAX calculent somme, moyenne, minimum et maximum en ignorant les valeurs NULL.

```
SELECT COUNT(*) AS nb_fournisseurs FROM fournisseurs;
SELECT COUNT(*) AS nb_articles FROM articles WHERE num_frs = 777;
SELECT SUM(pa_art) AS somme, AVG(pa_art) AS moyenne,
       MIN(pv_art) AS prix_min, MAX(pv_art) AS prix_max
FROM articles;
SELECT COUNT(DISTINCT cod_cli) AS nb_clients
FROM ventes
WHERE dat_vnt >= '2026-01-01' AND dat_vnt < '2026-01-09';
```

SUM(pa_art)/COUNT(*) ne reproduit AVG(pa_art) que si pa_art ne contient pas de NULL. Une somme de prix unitaires n’est pas une valeur de stock : celle-ci nécessiterait aussi des quantités.

Sous-requêtes

```
SELECT des_art FROM articles
WHERE pv_art = (SELECT MAX(pv_art) FROM articles);
SELECT num_art FROM articles
WHERE pv_art = (SELECT MIN(pv_art) FROM articles);
SELECT num_frs FROM articles
WHERE pa_art = (SELECT MAX(pa_art) FROM articles);
SELECT des_art FROM articles
WHERE pv_art > (SELECT AVG(pv_art) FROM articles);
```

Ces requêtes conservent les ex æquo. Un agrégat ne s’utilise pas directement dans le WHERE du même niveau de requête ; il peut apparaître dans une sous-requête, comme ici.

Tri

```
SELECT * FROM clients ORDER BY nom_cli ASC, prn_cli ASC;
SELECT * FROM clients ORDER BY prn_cli DESC;
SELECT * FROM articles ORDER BY pa_art DESC, pv_art ASC;
```

ASC est l’ordre croissant par défaut ; DESC l’ordre décroissant. Sans ORDER BY, l’ordre des résultats n’est pas garanti.

Application

Afficher les articles dont le prix dépasse la moyenne, du plus cher au moins cher. Compter les clients distincts ayant effectué au moins une vente en octobre 2026.

Construire une requête SQL : la méthode

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Partir de l’énoncé et du schéma

Méthode du support « Comment construire une requête SQL ». Relever les tables et leurs clés avant d’écrire. Les noms id et id_table sont des conventions possibles, pas des noms imposés.

1. Que veut-on afficher ? → SELECT

Repérer « afficher », « donner », « lister », « combien ». Choisir colonnes et calculs : COUNT, SUM, AVG, MIN, MAX. Nommer les résultats avec un alias ; utiliser DISTINCT pour supprimer les doublons du résultat.

2. Dans quelles tables ? → FROM

Lister les tables contenant les informations nécessaires. Ajouter une table intermédiaire si elle permet de relier deux autres tables. Une même table utilisée deux fois doit recevoir deux alias différents.

3. Comment relier les tables ? → WHERE

Écrire les conditions de jointure dans WHERE : table1.champX = table2.champX, en suivant les clés du schéma. Pour une chaîne simple de n tables, n − 1 liaisons relient les tables ; un schéma particulier peut imposer d’autres conditions.

4. Faut-il filtrer des lignes ? → WHERE

Repérer « dont », « de la classe X », « supérieur à », « en 2026 ». Ajouter les conditions avec AND ou OR ; mettre des parenthèses si nécessaire. Les textes et les dates littérales sont entre apostrophes ; LIKE utilise % et _. Ne pas placer de fonction d’agrégat directement dans WHERE au même niveau de requête.

5. Faut-il calculer par groupe ? → GROUP BY

Repérer « par classe », « pour chaque », « selon », avec une moyenne, un nombre ou un total. Mettre dans GROUP BY les colonnes affichées sans agrégat. Grouper sur l’identifiant, et pas seulement sur le nom : deux homonymes ne doivent pas fusionner.

6. Faut-il filtrer les groupes ? → HAVING

Repérer « dont la moyenne dépasse », « ayant plus de 3 ». Écrire le filtre sur l’agrégat dans HAVING. WHERE filtre les lignes avant le calcul ; HAVING filtre les groupes après le calcul.

7. Faut-il trier ? → ORDER BY

Repérer « trié par », « classé », « du plus grand au plus petit ». ASC est croissant ; DESC est décroissant. Le tri vient à la fin.

```
SELECT colonnes, fonction_agregat(colonne) AS resultat
FROM table1, table2
WHERE table1.champX = table2.champX AND condition_ligne
GROUP BY colonnes
HAVING fonction_agregat(colonne) > seuil
ORDER BY resultat DESC;
```

Ce dernier bloc est un gabarit à adapter : ses noms sont des repères. Les clauses facultatives sont omises lorsque l’énoncé ne les demande pas.

Construire une requête SQL : cas particuliers

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Reconnaître une sous-requête

Si une valeur doit d'abord être calculée pour permettre une comparaison, utiliser un SELECT entre parenthèses. Indices : « supérieur à la moyenne générale », « le plus cher », « qui ont au moins une… », « qui n'ont jamais… ». Choisir selon le schéma et le résultat attendu.

```
SELECT num_art, des_art, pv_art
FROM articles
WHERE pv_art > (SELECT AVG(pv_art) FROM articles);
```

Avec =, > ou <, la sous-requête doit produire une seule valeur. IN et NOT IN comparent à une liste. Pour NOT IN, exclure les valeurs NULL de la liste. Une sous-requête peut aussi intervenir dans HAVING pour comparer un agrégat à une valeur calculée.

Reconnaître une auto-jointure

Une même table intervient deux fois : « deux élèves de la même classe », « plus âgé que », « les paires de… ». Déclarer deux alias. Exemple : paires d'articles du même fournisseur dans commerce.

```
SELECT a1.num_art AS article1, a2.num_art AS article2
FROM articles AS a1, articles AS a2
WHERE a1.num_frs = a2.num_frs
  AND a1.num_art < a2.num_art;
```

L'inégalité stricte empêche d'associer un article avec lui-même et ne conserve qu'un ordre de chaque paire. Avec <> seul, on obtiendrait les deux ordres.

Construire une requête complète

Énoncé : afficher les clients dont les ventes dépassent 1000, avec leur total, du plus grand au plus petit. Une ligne de ventes contient une quantité et le prix unitaire réellement appliqué.

```
SELECT c.cod_cli, c.nom_cli,
       SUM(v.qte_vnt * v.prix_unitaire) AS total_ventes
FROM clients AS c, ventes AS v
WHERE c.cod_cli = v.cod_cli
GROUP BY c.cod_cli, c.nom_cli
HAVING SUM(v.qte_vnt * v.prix_unitaire) > 1000
ORDER BY total_ventes DESC;
```

Liste de contrôle avant de valider

- Chaque table nécessaire est présente, y compris une éventuelle table intermédiaire.
- Chaque liaison suit le schéma et figure dans WHERE ; aucune table n'est laissée isolée par oubli.
- Chaque alias est unique et utilisé de manière cohérente ; les colonnes ambiguës sont qualifiées.
- Les colonnes affichées sans agrégat figurent dans GROUP BY lorsqu'il y a regroupement.
- Aucun agrégat n'est appliqué directement aux lignes dans le WHERE du même niveau.
- Les filtres de groupes sont dans HAVING ; le tri est dans ORDER BY.
- Le résultat est vérifié sur quelques lignes : doublons, homonymes, valeurs NULL et ex æquo.

SCANNER LA FICHE
ou cliquer sur le QR code

GROUP BY, HAVING et jointures

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Regrouper puis filtrer les groupes

```
SELECT num_frs, COUNT(*) AS nb_articles
FROM articles GROUP BY num_frs;
SELECT DATE(dat_vnt) AS jour, SUM(qte_vnt) AS quantite_vendue
FROM ventes GROUP BY DATE(dat_vnt);
SELECT cod_cli, COUNT(*) AS nb_lignes_vente
FROM ventes GROUP BY cod_cli HAVING COUNT(*) <= 2;
SELECT num_art FROM ventes
GROUP BY num_art HAVING COUNT(*) > 2;
```

WHERE filtre les lignes avant le regroupement ; HAVING filtre les groupes après calcul. « Deux au plus » signifie <= 2, et non >= 2. Ce comptage concerne les lignes de vente, pas les unités ni les factures. Les clients sans vente ne figurent pas dans le regroupement de ventes.

Les colonnes sélectionnées hors agrégat doivent figurer dans GROUP BY, sauf dépendance fonctionnelle reconnue par le SGBD. Pour débuter, écrire explicitement toutes ces colonnes dans GROUP BY.

Jointure interne

```
SELECT a.num_art, f.nom_frs, f.tel_frs
FROM articles AS a, fournisseurs AS f
WHERE a.num_frs = f.num_frs;
SELECT DISTINCT a.des_art, a.col_art
FROM articles AS a, ventes AS v
WHERE a.num_art = v.num_art;
```

Les tables sont séparées par des virgules dans FROM. La condition de liaison est écrite dans WHERE sous la forme table1.champ = table2.champ. Les alias évitent les ambiguïtés ; DISTINCT évite ici de répéter un article vendu plusieurs fois.

Total des ventes par client

```
SELECT c.cod_cli, c.nom_cli,
       SUM(v.qte_vnt * v.prix_unitaire) AS total_ventes
FROM clients AS c, ventes AS v
WHERE c.cod_cli = v.cod_cli
GROUP BY c.cod_cli, c.nom_cli
HAVING SUM(v.qte_vnt * v.prix_unitaire) > 1000;
```

Absence de correspondance avec NOT IN

```
SELECT f.nom_frs, f.tel_frs FROM fournisseurs AS f
WHERE f.num_frs NOT IN (
  SELECT a.num_frs FROM articles AS a
  WHERE a.num_frs IS NOT NULL
);
```

La sous-requête renvoie les fournisseurs référencés par les articles. NOT IN conserve les autres. Exclure les valeurs NULL dans la sous-requête : sinon une valeur inconnue peut empêcher les résultats attendus. Dans le schéma commerce, articles.num_frs est déjà NOT NULL.

Fonctions SQL sur les chaînes

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Fonctions MySQL usuelles

```
SELECT CONCAT('Amine', ' ', 'Benali') AS nom_complet;
SELECT UPPER('Tunis'), LOWER('HTML');
SELECT LENGTH('Tunis') AS octets,
       CHAR_LENGTH('Tunis') AS caracteres;
SELECT SUBSTRING('abcdefghij', 2, 3) AS extrait; -- bcd
SELECT LEFT('abcdefghij', 2) AS debut;          -- ab
SELECT RIGHT('abcdefghij', 3) AS fin;           -- hij
SELECT TRIM('  STI  ') AS propre;
SELECT REPLACE('HTML et HTML', 'HTML', 'CSS') AS resultat;
```

LENGTH mesure des octets ; CHAR_LENGTH mesure des caractères. Avec un encodage multioctet comme UTF-8, les résultats peuvent différer. En SQL, SUBSTRING utilise une position initiale 1, contrairement aux indices de chaîne JavaScript qui commencent à 0.

Appliquer aux données

```
USE commerce;
SELECT cod_cli, CONCAT(prn_cli, ' ', nom_cli) AS identite
FROM clients ORDER BY nom_cli, prn_cli;
SELECT num_art, LEFT(des_art, 10) AS libelle_court
FROM articles;
SELECT * FROM fournisseurs
WHERE CHAR_LENGTH(nom_frs) = 6 AND nom_frs LIKE 'Ka%';
UPDATE clients SET nat_cli = UPPER(nat_cli);
```

L'UPDATE touche toutes les lignes : le tester d'abord avec SELECT. CONCAT retourne NULL si l'un de ses arguments est NULL ; COALESCE permet de remplacer une valeur absente.

```
SELECT CONCAT(prn_cli, ' ', nom_cli, ' - ',
              COALESCE(nat_cli, 'Non renseignée')) AS fiche
FROM clients;
```

Application

Afficher les trois premières lettres du nom de chaque fournisseur. Rechercher les noms qui contiennent six caractères et commencent par Ka. Comparer les longueurs en octets et en caractères d'un texte contenant « é ».

Fonctions SQL sur les dates et heures

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Lire et extraire

```
SELECT NOW() AS instant, CURRENT_DATE() AS jour;
SELECT YEAR('2026-10-21') AS annee,
       MONTH('2026-10-21') AS mois,
       DAY('2026-10-21') AS jour;
SELECT HOUR('12:30:45'), MINUTE('12:30:45'), SECOND('12:30:45');
SELECT DATE('2026-10-21 12:30:00') AS date_seule;
SELECT TIMESTAMP('2026-10-21', '12:30:00') AS instant;
```

NOW dépend du fuseau horaire de la session MySQL.

TIMESTAMP() est ici une fonction de construction, à distinguer du type de colonne TIMESTAMP et d'un timestamp Unix en secondes.

Ajouter, retrancher et comparer

```
SELECT DATE_ADD('2026-08-25', INTERVAL 3 MONTH); -- 2026-11-25
SELECT DATE_SUB('2026-10-21', INTERVAL 7 DAY);   -- 2026-10-14
SELECT DATEDIFF('2026-08-25', '2026-08-15');     -- 10
SELECT TIMESTAMPDIFF(YEAR, '2008-03-14', '2026-10-21'); -- 18
```

DATEDIFF(a,b) calcule a moins b en jours et ignore les heures. TIMESTAMPDIFF(unite,debut,fin) mesure des unités entières entre deux instants ; l'ordre des arguments diffère de celui de DATEDIFF.

Rechercher une période

```
SELECT * FROM ventes
WHERE dat_vnt >= '2026-10-01' AND dat_vnt < '2026-11-01';
SELECT DATE(dat_vnt) AS jour, SUM(qte_vnt) AS quantite
FROM ventes GROUP BY DATE(dat_vnt) ORDER BY jour;
```

Pour un filtre de période, comparer la colonne directement à des bornes peut permettre un meilleur usage des index que YEAR(dat_vnt) ou MONTH(dat_vnt). Pour un regroupement journalier, DATE(dat_vnt) exprime explicitement l'intention.

Application

Calculer une date de retour à partir de date_loc et duree_loc dans gestion_location. Compter les clients distincts ayant acheté pendant les sept dernières journées civiles complètes.

Jointures dans WHERE et mises à jour croisées

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Écrire une jointure interne

```
SELECT articles.num_art, fournisseurs.nom_frs
FROM articles, fournisseurs
WHERE articles.num_frs = fournisseurs.num_frs;
```

La jointure relie des lignes de tables différentes. La liaison compare ici la clé étrangère articles.num_frs à la clé primaire fournisseurs.num_frs. Oublier cette condition produit un produit cartésien : chaque article est associé à chaque fournisseur.

Utiliser des alias et ajouter un filtre

```
SELECT a.num_art, a.des_art, f.nom_frs
FROM articles AS a, fournisseurs AS f
WHERE a.num_frs = f.num_frs
  AND a.pv_art > 50
ORDER BY a.des_art ASC;
```

Après la déclaration des alias, qualifier les colonnes avec ces alias. AND réunit la condition de liaison et la condition de sélection. Pour relier trois tables en chaîne, il faut généralement deux liaisons ; vérifier les clés du schéma plutôt que d’ajouter une égalité au hasard.

UPDATE utilisant deux tables

Exemple indépendant : appliquer des tarifs transmis aux articles correspondants. Créer la table des nouveaux prix avant la mise à jour.

```
CREATE TABLE nouveaux_prix (
  num_art INT PRIMARY KEY,
  nouveau_prix DECIMAL(10,3) NOT NULL CHECK (nouveau_prix > 0)
);
INSERT INTO nouveaux_prix VALUES (501, 18.750);
SELECT a.num_art, a.pv_art, n.nouveau_prix
FROM articles AS a, nouveaux_prix AS n
WHERE a.num_art = n.num_art;
UPDATE articles AS a, nouveaux_prix AS n
SET a.pv_art = n.nouveau_prix
WHERE a.num_art = n.num_art;
```

La clé primaire de nouveaux_prix évite plusieurs prix candidats pour un même article. Vérifier les correspondances avec SELECT, conserver une sauvegarde puis contrôler le nombre de lignes modifiées. L’exemple suppose que l’article 501 existe pour obtenir une mise à jour.

Application

Afficher les articles avec leur fournisseur, puis seulement ceux dont le prix dépasse 50. Préparer une mise à jour croisée sur une copie de la base. Distinguer les commandes de définition (LDD) et de manipulation (LMD).

Principe client-serveur et environnement

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Une page dynamique

Le navigateur envoie une requête HTTP au serveur. Le serveur exécute le script PHP, peut interroger MySQL, puis renvoie une réponse, souvent HTML. Le navigateur reçoit cette réponse, pas le code source PHP lorsque le serveur est correctement configuré.

Navigateur → requête HTTP → serveur Web + PHP → MySQL si nécessaire → réponse HTML → navigateur.

Le traitement côté serveur permet de centraliser les règles et l'accès aux données. Il ne garantit ni un téléchargement plus rapide dans tous les cas, ni l'absence de problèmes de compatibilité du HTML produit.

Environnement de travail

Utiliser une version PHP 8.x maintenue, un serveur Web configuré pour PHP et un serveur MySQL. Un environnement local intégré peut fournir ces composants. phpMyAdmin est une interface d'administration de MySQL, pas le serveur de base de données lui-même.

Enregistrer les scripts avec l'extension .php dans le dossier servi par le serveur. Ce dossier dépend de la configuration : il ne s'appelle pas nécessairement www. Ne pas ouvrir le fichier PHP directement avec file://.

Première page

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Première page PHP</title>
</head>
<body>
  <h1><?php echo "Bienvenue au Lycée Rafèha"; ?></h1>
</body>
</html>
```

Syntaxe et commentaires

Commencer le code par <?php. Les instructions se terminent par un point-virgule ; // et /* ... */ servent aux commentaires. Dans un fichier contenant exclusivement du PHP, on omet généralement la balise fermante pour éviter des sorties accidentelles.

PHP signifie « PHP: Hypertext Preprocessor ». Le langage est à typage dynamique et sensible à la casse pour les noms de variables.

Convention du cours

Les accès aux bases utilisent exclusivement l'API MySQLi procédurale, sous la forme mysqli_*. Le parcours des tableaux est effectué avec des boucles for ou while.

Variables, types, constantes et opérateurs

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Variables et constantes

```
<?php
$nom = "Amal";
$age = 18;
$moyenne = 15.5;
$admis = true;
$absence = null;
define("ETABLISSEMENT", "Lycée Rafèha");
echo ETABLISSEMENT . " : " . $nom;
```

Une variable commence par $, puis une lettre ou un soulignement, suivis éventuellement de chiffres. $nom et $Nom désignent deux variables différentes. Les constantes ne commencent pas par $.

Les types usuels sont int, float, string, bool, array et null. Une nouvelle affectation peut changer le type d'une variable non typée, mais les opérations et les paramètres restent soumis aux règles de type de PHP.

Fonctions sur les variables

```
<?php
$x = 2.3;
echo gettype($x);        // double (nom historique)
settype($x, "integer");
echo $x;                 // 2
$present = isset($x);    // true
unset($x);
$present = isset($x);    // false
$valeur = null;
$present = isset($valeur); // false
```

isset retourne false si la variable n'existe pas ou vaut null. Un cast comme (int)$x effectue une conversion. Il ne remplace pas la validation d'une entrée utilisateur.

Opérateurs

+ - * / % ** : calcul ; . : concaténation ; = : affectation ; += et .= : mise à jour. == compare avec conversions éventuelles ; === compare valeur et type. Utiliser &&, || et ! pour les expressions logiques usuelles.

```
<?php
$prix = 25.5;
$quantite = 3;
$total = $prix * $quantite;
$message = "Total : " . $total . " DT";
echo $message;
```

Application

Définir une constante TAUX_TVA, calculer un prix TTC et afficher un message. Expliquer la différence entre une conversion de type et le contrôle de validité d'une saisie.

SCANNER LA FICHE
ou cliquer sur le QR code

Formulaires : GET, POST, isset et empty

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Du navigateur au serveur

Le navigateur envoie les champs selon leurs attributs name. GET place les paramètres dans l'URL ; POST les place dans le corps de la requête. POST ne chiffre pas les données : HTTPS protège le transport. Une valeur simple reçue par GET ou POST est normalement une chaîne ; des paramètres structurés peuvent aussi produire des tableaux.

```
<form action="traitement.php" method="post">
  <label for="age">Âge</label>
  <input id="age" name="age" type="number" min="0" max="120">
  <button type="submit">Envoyer</button>
</form>
```

Vérifier la présence et le type avant de convertir

```
<?php
if (!isset($_POST["age"]) || !is_string($_POST["age"])) {
    die ("Âge manquant ou invalide.");
}
$texte = trim($_POST["age"]);
if ($texte === "" || !is_numeric ($texte) || strlen($texte) > 3) {
    die ("Un entier positif ou nul est attendu.");
}
$age = (int)$texte;
if ($age > 120) { die ("Âge hors limites."); }
echo "Âge validé : " . $age;
```

La donnée affichée est ici un entier validé, pas le texte brut reçu. Pour GET, adapter le formulaire et lire $_GET["age"]. La validation JavaScript ne remplace jamais celle du serveur.

isset et empty ne répondent pas à la même question

```
<?php
$valeur = "0";
echo isset($valeur) ? "présente" : "absente"; // présente
echo empty($valeur) ? "vide au sens PHP" : "non vide"; // vide au sens PHP
```

isset est vrai si la variable existe et n'est pas null. empty est vrai notamment pour une variable absente, null, false, 0, 0.0, "", "0" ou un tableau vide. Un champ texte vide peut être envoyé : il est présent mais vide. Ne pas rejeter zéro avec empty lorsque zéro est autorisé.

Limite volontaire du cours

Ces exemples n'affichent pas librement une saisie textuelle dans du HTML et ne la concatènent pas dans une requête SQL. Leur simplification ne constitue pas une protection générale pour un site public.

Contre-exemple barré : ne pas utiliser

```
<?php
// ~~var_dump($valeur)~~; // Instruction exclue des exemples du cours.
```

Pour lire un booléen, afficher explicitement "true" ou "false" avec une condition, comme ci-dessus.

Conditions, boucles et fonctions

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Conditions

```
<?php
$note = 14;
if ($note < 0 || $note > 20) {
    echo "Note invalide";
} elseif ($note >= 10) {
    echo "Admis";
} else {
    echo "Non admis";
}
```

switch fonctionne avec case, break et default. En PHP, switch utilise des comparaisons non strictes : ne pas lui attribuer le comportement strict du switch JavaScript.

Boucles

```
<?php
$somme = 0;
for ($i = 1; $i <= 10; $i++) {
    $somme += $i;
}
echo $somme; // 55
$n = 10;
$factorielle = 1;
while ($n > 0) {
    $factorielle *= $n;
    $n--;
}
$i = 0;
do {
    echo $i . "<br>";
    $i++;
} while ($i < 3);
```

Initialiser les accumulateurs avant la boucle. while teste avant le bloc ; do...while teste après et s’exécute au moins une fois.

Fonction et passage de paramètres

```
<?php
function calculerMoyenne($somme, $effectif) {
    if ($effectif <= 0) { return null; }
    return $somme / $effectif;
}
echo calculerMoyenne(42, 3); // 14
```

return termine l’appel et fournit une valeur. Sans retour explicite, une fonction renvoie null. Les paramètres sont transmis par valeur par défaut. Le préfixe & permet un passage par référence, à utiliser uniquement si la modification de la variable de l’appelant est voulue. Les variables d’une fonction ont une portée locale.

Tableaux, parcours avec for et tris

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Tableaux indexés et associatifs

```
<?php
$notes = [12, 15, 9];
$notes[] = 16;
$ventes = ["lundi" => 7, "mardi" => 5, "jeudi" => 9];
echo $notes[0]; // 12
echo count($notes); // 4
echo $ventes["mardi"]; // 5
```

array(...) et [...] construisent un tableau. Les clés peuvent être entières ou textuelles. Après l'affectation $tab[10] = "valeur", un ajout par $tab[] utilise l'indice entier suivant, 11, si aucun indice supérieur n'existe.

Parcourir avec une boucle classique

```
<?php
for ($i = 0; $i < count($notes); $i++) {
    echo $notes[$i] . "<br>";
}
$cles = array_keys($ventes);
for ($i = 0; $i < count($cles); $i++) {
    $jour = $cles[$i];
    echo $jour . " : " . $ventes[$jour] . " unités<br>";
}
```

Un parcours de 0 à count − 1 suppose des indices contigus. Pour un tableau associatif ou des indices non contigus, array_keys fournit les clés à parcourir. Les textes affichés ici sont des constantes de cours.

Trier puis parcourir

sort et rsort trient les valeurs et réindexent les clés numériques. asort et arsort trient les valeurs en conservant les clés. ksort et krsort trient selon les clés.

```
<?php
sort($notes, SORT_NUMERIC);
asort($ventes, SORT_NUMERIC);
$cles = array_keys($ventes);
for ($i = 0; $i < count($cles); $i++) {
    $jour = $cles[$i];
    echo $jour . " : " . $ventes[$jour] . "<br>";
}
```

Tableau à plusieurs dimensions

$eleves = [["nom" => "Amal", "note" => 16], ["nom" => "Ali", "note" => 14]] ; accéder à la première note par $eleves[0]["note"]. Une boucle for sur les indices contigus permet de lire chaque ligne.

SCANNER LA FICHE
ou cliquer sur le QR code

Chaînes et fonctions usuelles

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Fonctions de base sur les chaînes

trim enlève les blancs aux extrémités ; strlen mesure une longueur en octets ; substr extrait à partir d'un indice commençant à 0 ; str_replace remplace toutes les occurrences recherchées.

```
<?php
$texte = trim("  Bonjour Tunis  ");
echo strlen("Bonjour");       // 7
echo substr("Informatique", 3, 4); // orma
echo str_replace("chat", "chien", "Le chat dort."); // Le chien dort.
```

Rechercher et comparer

```
<?php
$position = strpos("Bonjour tout le monde", "tout");
echo $position; // 8
$debut = strpos("Bonjour", "Bon");
if ($debut !== false) { echo "Trouvé à " . $debut; } // 0
echo strcmp("abc", "abc"); // 0
```

strpos renvoie false si le texte est absent, mais peut renvoyer 0 s'il commence au début : comparer strictement à false. strcmp renvoie 0 pour deux chaînes égales, un nombre négatif ou positif selon leur ordre.

Découper et assembler

```
<?php
$couleurs = explode(",", "rouge,vert,bleu");
echo $couleurs[1]; // vert
$noms = ["Ali", "Sami", "Amel"];
echo implode(" - ", $noms); // Ali - Sami - Amel
```

explode découpe selon un séparateur non vide ; implode assemble les éléments avec un séparateur. Les textes affichés ici sont des constantes du cours, pas des données reçues.

Codes, casse et inversion

```
<?php
echo chr(65);                  // A
echo ord("A");                 // 65
echo strrev("STI");            // ITS
echo strtolower("Bonjour STI"); // bonjour sti
echo strtoupper("Bonjour STI"); // BONJOUR STI
```

chr construit un caractère à partir d'un octet ; ord lit le premier octet. Les exemples utilisent l'ASCII. strlen, substr et strrev travaillent sur des octets ; la casse ci-dessus concerne l'ASCII. Pour l'UTF-8 accentué, des fonctions multioctets adaptées peuvent être nécessaires.

Application

Découper une liste de mots, rechercher une occurrence et expliquer le résultat 0. Ne pas confondre la conversion de casse et la validation d'une saisie.

SCANNER LA FICHE
ou cliquer sur le QR code

Dates et heures : timestamp et checkdate

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Pourquoi manipuler les dates ?

Un site dynamique peut enregistrer la date système, afficher une date dans un autre format, ajouter une période, comparer des échéances ou traiter des champs DATE et DATETIME.

Un timestamp Unix est un entier représentant un nombre de secondes depuis le 1er janvier 1970 à 00:00:00 UTC. Il désigne un instant ; le fuseau horaire intervient lorsqu'on le transforme en date locale. PHP utilise le type int, dont la capacité dépend de la plateforme. Préférer un environnement 64 bits pour une large plage de dates.

1. La fonction checkdate

Signature : checkdate(int $month, int $day, int $year): bool

Elle renvoie true si la date est valide, sinon false. L'ordre est mois, jour, année, et non l'ordre français jour, mois, année.

• Année : de 1 à 32767 inclus.

• Mois : de 1 à 12 inclus.

• Jour : doit exister dans le mois et l'année considérés ; les années bissextiles sont prises en compte.

```
<?php
$j = 30;
$m = 11;
$a = 2026;
if (checkdate($m, $j, $a)) {
    $jour = ($j < 10 ? "0" : "") . $j;
    $mois = ($m < 10 ? "0" : "") . $m;
    $d = $jour . "/" . $mois . "/" . $a;
} else {
    $d = "Données invalides";
}
echo $d; // 30/11/2026
```

Tests complémentaires

checkdate(2, 29, 2026) renvoie false ; checkdate(2, 29, 2028) renvoie true. Cette fonction vérifie une date civile ; elle ne crée pas de timestamp et ne valide pas à elle seule une heure.

Application

Contrôler une date de naissance saisie en trois champs. Après la validité civile, vérifier aussi les règles de gestion : par exemple, la date ne doit pas être future.

Dates et heures : date et getdate

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

La fonction date

Signature : date(string $format, ?int $timestamp = null): string. Elle retourne une chaîne ; sans timestamp, elle utilise l’instant courant, selon le fuseau configuré.

```
<?php

echo date("Y-m-d H:i:s"); // date et heure actuelles
```

Formats du récapitulatif : attention à la casse

```
<?php
$instant = mktime(16, 50, 0, 10, 21, 2026);
echo date("Y-m-d", $instant); // 2026-10-21
echo date("y-m-d", $instant); // 26-10-21
echo date("d-m-Y", $instant); // 21-10-2026
echo date("d/m/Y", $instant); // 21/10/2026
echo date("H:i:s", $instant); // 16:50:00
```

Y : année sur 4 chiffres ; y : sur 2. m : mois sur 2 chiffres ; n : sans zéro initial. d : jour sur 2 chiffres ; j : sans zéro initial. H : heure sur 24 h ; i : minutes ; s : secondes. Les tirets et barres obliques sont littéraux. Ne pas confondre m et i.

La fonction getdate

Signature : getdate(?int $timestamp = null): array. Elle renvoie un tableau décrivant la date locale ; sans argument, elle utilise l’instant courant.

```
<?php

$a = getdate();
echo "Jour : " . $a["mday"] . "<br>";
echo "Mois : " . $a["mon"] . "<br>";
echo "Année : " . $a["year"] . "<br>";
echo "Heure : " . $a["hours"] . ":" . $a["minutes"];
```

Clés et valeurs de getdate

• seconds : 0–59 ; minutes : 0–59 ; hours : 0–23.

• mday : 1–31 ; mon : 1–12 ; year : année.

• wday : 0 (dimanche) à 6 ; yday : 0–365.

• weekday et month : noms anglais du jour et du mois ; clé numérique 0 : timestamp.

La clé weekday s’écrit en minuscules. Les mois de getdate commencent à 1, contrairement à getMonth() en JavaScript. Pour un affichage reproductible, fournir un timestamp fixe ; pour une date civile saisie, employer checkdate avant de la construire.

SCANNER LA FICHE
ou cliquer sur le QR code

Dates et heures : time et mktime

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

4. La fonction time

Signature : time(): int

Elle retourne le timestamp Unix courant, en secondes.

```
<?php

$a = time();
$b = date("d/m/Y H:i:s", $a);
echo $a . " = " . $b;
```

Le nombre renvoyé évolue à chaque seconde. Il ne faut pas présenter une valeur fixe comme résultat permanent de cet exemple.

5. La fonction mktime

Signature PHP 8 : mktime(int $hour, ?int $minute = null, ?int $second = null, ?int $month = null, ?int $day = null, ?int $year = null): int|false

Elle construit un timestamp à partir de composantes interprétées dans le fuseau local. L'heure est obligatoire en PHP 8 ; les autres paramètres omis ou null prennent les valeurs de la date courante. Fournir tous les paramètres pour un exemple reproductible.

```
<?php

$ns = mktime(12, 30, 0, 10, 21, 2026);
if ($ns === false) {
    die ("Date non représentable.");
}
echo "Timestamp : " . $ns . "<br>";
echo "Date : " . date("d-m-Y H:i:s", $ns);
// Date : 21-10-2026 12:30:00
```

Normalisation des dépassements

```
<?php
$suivant = mktime(12, 30, 0, 10, 32, 2026);
if ($suivant !== false) {
    echo date("d-m-Y H:i:s", $suivant);
    // 01-11-2026 12:30:00
}
```

mktime normalise les composantes hors plage ; ce n'est donc pas un validateur. Pour une saisie, vérifier d'abord la date avec checkdate et l'heure avec des bornes appropriées.

Correction importante

L'ancien paramètre is_dst n'existe plus : il a été supprimé en PHP 7. Ne pas transmettre de septième argument. Les règles d'heure d'été éventuelles proviennent du fuseau configuré ; l'ancien texte sur « l'heure d'hiver » était incorrect.

Dates et heures : strtotime et calculs

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

6. La fonction strtotime

Signature : strtotime(string $datetime, ?int $baseTimestamp = null): int|false

Elle analyse une expression de date en anglais ou certains formats numériques. Les expressions relatives utilisent le timestamp de référence, ou l'instant courant lorsqu'il est omis. Le fuseau par défaut intervient si l'expression n'en précise pas.

```
<?php

$base = mktime(12, 0, 0, 10, 21, 2026);
if ($base === false) { die ("Référence invalide."); }
$expressions = [
    "now", "10 September 2026", "+1 day", "+1 week",
    "+1 week 2 days 4 hours 2 seconds"
];
for ($i = 0; $i < count($expressions); $i++) {
    $expression = $expressions[$i];
    $ts = strtotime($expression, $base);
    if ($ts === false) {
        echo "Expression invalide<br>";
        continue;
    }
    echo $expression
       . " : " . date("Y-m-d H:i:s", $ts)
       . " = " . $ts . " secondes<br>";
}
```

Avec cette référence, +1 day donne 2026-10-22 12:00:00 ; +1 week donne 2026-10-28 12:00:00 ; la dernière expression donne 2026-10-30 16:00:02. « 10 September 2026 » désigne le 10 septembre à minuit.

Ajouter une période à une date

```
<?php
$depart = strtotime("2026-11-30 09:00:00");
if ($depart === false) { die ("Date de départ invalide."); }
$echeance = strtotime("+7 days", $depart);
if ($echeance !== false) {
    echo date("d/m/Y H:i:s", $echeance);
    // 07/12/2026 09:00:00
}
```

Limites

Tester === false : le timestamp 0 est valide. Éviter les dates ambiguës comme 01/02/2026 ; préférer 2026-02-01. strtotime peut normaliser certaines dates civiles impossibles : utiliser checkdate pour une validation stricte. Un mois ajouté en fin de mois peut déborder ; une journée civile ne vaut pas toujours 86400 secondes dans tous les fuseaux.

MySQLi procédural : connexion et requête fixe

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Uniquement les fonctions procédurales retenues

MySQLi relie PHP à MySQL. Ordre habituel : connexion → choix de la base → requête → lecture du résultat → fermeture. Les exemples supposent un serveur local de cours configuré, l'extension mysqli active et les bases déjà créées.

```
<?php
$cnx = mysqli_connect("localhost", "utilisateur_cours",
                      "mot_de_passe", "commerce");
if (!$cnx) { die ("Connexion impossible."); }
$res = mysqli_query($cnx,
    "SELECT num_art, pv_art FROM articles ORDER BY num_art"
);
if ($res === false) { die ("Requête impossible."); }
echo "Nombre d'articles : " . mysqli_num_rows($res) . "<br>";
while ($ligne = mysqli_fetch_assoc($res)) {
    echo (int)$ligne["num_art"] . " : "
       . (float)$ligne["pv_art"] . " DT<br>";
}
mysqli_close($cnx);
```

Les identifiants sont des exemples à adapter. Les sorties variables sont ici numériques ; aucun texte libre de la base n'est injecté dans le HTML.

Deux formes de connexion

```
<?php
// Forme indépendante : connexion, puis sélection de la base.
$cnx = mysqli_connect("localhost", "utilisateur_cours", "mot_de_passe");
if (!$cnx) { die ("Connexion impossible."); }
if (!mysqli_select_db($cnx, "commerce")) {
    die ("Base indisponible.");
}
mysqli_close($cnx);
```

La première forme sélectionne la base par le quatrième argument. mysqli_select_db permet aussi de changer de base sur une connexion ouverte. Après mysqli_close, ne plus utiliser cette connexion.

Erreurs : connaître la configuration

Les tests de false s'appliquent lorsque la configuration renvoie false. Depuis PHP 8.1, la configuration MySQLi par défaut peut interrompre le script lors d'une erreur au lieu de renvoyer false. Ces exemples ne gèrent pas tous les défauts de serveur : le professeur prépare l'environnement local et consulte ses diagnostics. Le script ne modifie pas le mode de signalement et ne traite pas les exceptions.

Encodage

Configurer l'environnement de cours et la connexion en UTF-8 adapté à la base. La création d'une base en utf8mb4 ne garantit pas à elle seule l'encodage de la connexion. Ce réglage relève ici de l'environnement, pas d'une fonction supplémentaire dans les exemples.

include, require et organisation des fichiers

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Réutiliser un fichier de connexion

include et require insèrent et exécutent un fichier PHP. Ils évitent de recopier les mêmes instructions dans chaque page. Les noms de fichiers inclus sont fixes dans ce cours : ne pas construire un chemin avec une saisie utilisateur.

```
<?php
// connexion.php
$cnx = mysqli_connect("localhost", "utilisateur_cours",
                      "mot_de_passe", "commerce");
if (!$cnx) { die ("Connexion impossible."); }
```

```
<?php
// liste.php
require "connexion.php";
$res = mysqli_query($cnx, "SELECT num_art FROM articles");
if ($res === false) { die ("Requête impossible."); }
echo mysqli_num_rows($res) . " article(s).";
mysqli_close($cnx);
```

Que se passe-t-il si le fichier manque ?

```
<?php
// Démonstration indépendante : fichier volontairement absent.
echo "Avant<br>";
include "fichier_absent.php";
echo "Après include<br>";
```

Si le fichier est introuvable, include produit un avertissement ; le script peut continuer. Avec require, l'échec arrête normalement le script. Une erreur produite à l'intérieur d'un fichier trouvé peut aussi interrompre l'exécution : la règle « continue » concerne l'échec du chargement par include, pas toute erreur possible.

Les variantes once

```
<?php
include_once "outils.php";
require_once "connexion.php";
```

include_once et require_once évitent de charger une seconde fois le même fichier. Les fichiers de cet exemple doivent exister. Ne pas fermer une connexion dont les pages suivantes ont encore besoin.

Bien organiser le travail

Une page de traitement reçoit et valide ; une page de lecture interroge puis affiche ; le fichier de connexion contient les paramètres communs. En environnement réel, ne pas publier les mots de passe dans le dépôt. Les exercices ici ne comprennent pas l'authentification ni les autorisations d'un service public.

MySQLi : SELECT, INSERT, UPDATE et DELETE

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Jeu d’essai du récapitulatif

Exercice indépendant à créer dans une base de test. Le champ id est attribué automatiquement. Le nom ci-dessous est une constante du cours, pas une saisie concaténée.

```
CREATE TABLE etudiant (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nom VARCHAR(50) NOT NULL,
  age INT NOT NULL CHECK (age BETWEEN 0 AND 120)
);
INSERT INTO etudiant (nom, age) VALUES ('Ali', 17), ('Amal', 18);
```

Insertion fixe et identifiant créé

```
<?php
// $cnx ouverte sur la base de test contenant etudiant.
$ok = mysqli_query($cnx,
    "INSERT INTO etudiant (nom, age) VALUES ('Sami', 17)"
);
if ($ok === false) { die ("Insertion impossible."); }
echo mysqli_affected_rows($cnx) . " ligne(s) ajoutée(s).<br>";
echo "Identifiant créé : " . mysqli_insert_id($cnx);
```

Autre écriture possible sur cette structure : INSERT INTO etudiant VALUES (NULL, 'Sami', 17). Lister les colonnes reste préférable. Ne pas demander l’identifiant AUTO_INCREMENT au formulaire. Réutiliser un identifiant existant peut provoquer un doublon.

Texte, nombre et date dans une requête fixe

```
<?php
$sql = "SELECT id, age FROM etudiant WHERE nom = 'Ali'";
$res = mysqli_query($cnx, $sql);
if ($res === false) { die ("Lecture impossible."); }
while ($ligne = mysqli_fetch_assoc($res)) {
    echo (int)$ligne["id"] . " : " . (int)$ligne["age"] . " ans<br>";
}
```

Les littéraux de texte et de date prennent des apostrophes SQL ; un nombre peut s’écrire sans apostrophes. Les guillemets doubles extérieurs délimitent la chaîne PHP. Cette règle de syntaxe n’autorise pas à concaténer une saisie textuelle dans SQL.

Modifier et supprimer sur une copie

```
<?php
$ok = mysqli_query($cnx, "UPDATE etudiant SET age = 18 WHERE id = 1");
if ($ok === false) { die ("Modification impossible."); }
echo mysqli_affected_rows($cnx) . " ligne(s) modifiée(s).";
$ok = mysqli_query($cnx, "DELETE FROM etudiant WHERE id = 2");
if ($ok === false) { die ("Suppression impossible."); }
mysqli_close($cnx);
```

Sans WHERE, toutes les lignes sont visées. true signifie que la requête a réussi ; cela ne prouve pas qu’une ligne a changé. Ne pas enchaîner ces exercices destructifs sur le jeu utilisé pour vérifier les autres réponses.

SCANNER LA FICHE
ou cliquer sur le QR code

Conversions, guillemets et fonctions arithmétiques

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Convertir n’est pas valider

```
<?php
echo (int)12.8;        // 12
echo (float)"12.8";    // 12.8
echo (string)25;       // chaîne "25"
echo ((bool)"0" ? "true" : "false");   // false
echo (int)"12.8abc";  // 12 : le suffixe n’est pas validé
echo (is_numeric("12.8") ? "true" : "false");    // true
echo (is_numeric("12.8abc") ? "true" : "false"); // false
```

is_numeric vérifie une représentation numérique, mais accepte aussi des réels et la notation scientifique. Pour un entier composé uniquement de chiffres, utiliser is_numeric sur une chaîne, puis vérifier longueur et bornes avant conversion.

Conversion automatique et comparaison

```
<?php
$a = "5"; $b = 5;
echo $a + $b;       // 10 : calcul numérique
echo $a . $b;       // 55 : concaténation
echo ($a == $b ? "true" : "false"); // true
echo ($a === $b ? "true" : "false");// false : types différents
```

En PHP 8, "abc" == 0 est faux. Convertir explicitement les données validées avant de calculer.

Apostrophes, guillemets et entier vers chaîne

```
<?php
$nom = "Ali";
echo 'Bonjour $nom';       // Bonjour $nom
echo "Bonjour $nom";       // Bonjour Ali
echo "Le nom est '$nom'";  // Le nom est 'Ali'
$n = 42;
$a = (string)$n;
$b = strval($n);
$c = "" . $n;
$d = "$n";
```

Les quatre formes produisent "42". Les guillemets doubles interprètent variables, \n et \t ; les apostrophes ne remplacent pas les variables. Le nom est une constante du cours.

Fonctions arithmétiques du récapitulatif

```
<?php
echo abs(-15);       // 15
echo sqrt(81);       // 9
echo round(3.14159, 2); // 3.14
echo rand(1, 6);     // entier de 1 à 6, bornes incluses
echo floor(4.8);     // 4
echo ceil(4.2);      // 5
```

rand sert ici à une simulation, pas à un secret. sqrt d’un négatif n’est pas réel ; les flottants peuvent être approchés. round renvoie un nombre.

SCANNER LA FICHE
ou cliquer sur le QR code

MySQLi : lecture des résultats et saisie numérique

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Résultat de mysqli_query

Une requête SELECT réussie fournit un résultat à parcourir. INSERT, UPDATE et DELETE réussis renvoient true. Selon la configuration, un échec renvoie false ou interrompt le script ; les contrôles ci-dessous ne sont pas une gestion complète des erreurs de connexion.

```
<?php
require "connexion.php"; // ouvre commerce
$res = mysqli_query($cnx,
    "SELECT num_art, pv_art FROM articles ORDER BY num_art"
);
if ($res === false) { die ("Lecture impossible."); }
echo mysqli_num_rows($res) . " article(s).<br>";
while ($ligne = mysqli_fetch_assoc($res)) {
    echo (int)$ligne["num_art"] . " : " . (float)$ligne["pv_art"] . "<br>";
}
$res = mysqli_query($cnx,
    "SELECT num_art, pv_art FROM articles ORDER BY num_art"
);
if ($res === false) { die ("Lecture impossible."); }
while ($ligne = mysqli_fetch_row($res)) {
    echo (int)$ligne[0] . " : " . (float)$ligne[1] . "<br>";
}
mysqli_close($cnx);
```

mysqli_fetch_assoc utilise les noms des colonnes ; mysqli_fetch_row utilise 0, 1… dans l'ordre du SELECT. Chaque appel avance d'une ligne ; null marque la fin. Relancer la requête pour recommencer le parcours. Les valeurs reçues peuvent notamment être des chaînes ou null ; selon le protocole et la configuration, des nombres peuvent aussi être fournis comme types numériques PHP.

Utiliser un code reçu, exclusivement numérique

```
<?php
if (!isset($_GET["code"]) || !is_string($_GET["code"])) {
    die ("Code manquant.");
}
$texte = trim($_GET["code"]);
if (!is_numeric ($texte) || strlen($texte) > 9) {
    die ("Code invalide.");
}
$code = (int)$texte;
if ($code < 1) { die ("Code invalide."); }
require "connexion.php";
$res = mysqli_query($cnx,
    "SELECT num_art, pv_art FROM articles WHERE num_art = " . $code
);
if ($res === false) { die ("Lecture impossible."); }
echo mysqli_num_rows($res) . " article(s) trouvé(s).";
mysqli_close($cnx);
```

Seul l'entier reconstruit est inséré dans SQL, jamais la chaîne reçue. Ne pas transposer cet exemple à un nom ou à une adresse libre. mysqli_num_rows compte les lignes du résultat ; SELECT COUNT(*) produit généralement une seule ligne contenant le compteur, à lire avec une fonction fetch.

Niveaux d’erreur PHP : tableau et exemples

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Tableau enrichi avec exemples

Les exemples ci-dessous sont volontairement fautifs et indépendants. Ils servent à reconnaître un diagnostic, pas à être réunis dans un script à exécuter.

Type	Gravité	Effet	Exemple de code fautif	Message affiché (indicatif)
Parse error	Syntaxe invalide	Le fichier concerné n’est pas exécuté.	`echo "Bonjour"` `echo "Suite";`	`Parse error: syntax error, unexpected token "echo", expecting "," or ";"`
Fatal error	Erreur grave	Le script s’arrête si l’erreur n’est pas traitée.	`afficher();` `// fonction jamais définie`	`Fatal error: Uncaught Error: Call to undefined function afficher()`
Warning	Problème non bloquant par défaut	Le script peut continuer.	`include("menu.php");` `// fichier inexistant`	`Warning: include(menu.php): Failed to open stream: No such file or directory`
Notice / Warning (variable)	Variable non définie	Le script peut continuer.	`echo $nom;` `// $nom non définie`	`PHP 8 : Warning: Undefined variable $nom` `PHP 7 : Notice: Undefined variable: nom`
Deprecated	Fonction ou usage obsolète	Le script continue généralement ; corriger l’usage.	`strlen(null);` `// PHP 8.1+, mode coercitif`	`Deprecated: strlen(): Passing null to parameter #1 ($string) of type string is deprecated`

Lire les messages correctement

Les messages sont indicatifs : leur formulation, le nom du fichier et le numéro de ligne dépendent de la version, du contexte et de la configuration. Les avertissements et notices laissent généralement continuer le script avec la configuration habituelle ; un gestionnaire personnalisé peut modifier ce comportement.

Précision sur l’exemple d’obsolescence

mysqli_connect accepte un nom d’hôte null : ce n’est pas un bon exemple de paramètre non nullable. Le tableau utilise donc strlen(null), qui peut produire E_DEPRECATED depuis PHP 8.1 en mode coercitif. En mode de typage strict, un argument incompatible peut produire TypeError.

Pour approfondir

La fiche suivante explique les niveaux E_PARSE, E_ERROR, E_WARNING, E_NOTICE et E_DEPRECATED, ainsi que Error, TypeError, les erreurs logiques et la méthode de recherche.

Comprendre les erreurs PHP et les diagnostics

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Erreur de syntaxe — E_PARSE

Une parenthèse, un guillemet ou un point-virgule manquant peut empêcher l'analyse du fichier. Le script concerné ne commence alors pas son exécution. La ligne annoncée peut se trouver après la faute réelle : relire aussi la ligne précédente.

```
<?php
// Contre-exemple à ne pas décommenter : echo "Bonjour"
// Correction :
echo "Bonjour";
```

Erreur fatale — E_ERROR ou erreur non traitée

Une fonction inexistante peut produire Error ; un mauvais type peut produire TypeError. Non traitées, ces erreurs interrompent le script. Un fichier indispensable absent peut aussi arrêter l'exécution. PHP moderne distingue ces objets des niveaux historiques ; le diagnostic dépend de la version et de la configuration.

```
<?php
// Exemple volontairement non exécuté : fonction_inconnue();
// require "fichier_indispensable_absent.php"; arrêterait le script.
```

Avertissement — E_WARNING

Un avertissement signale un problème sans arrêter systématiquement le script. include sur un fichier introuvable peut avertir puis laisser continuer. Une variable non définie produit généralement un avertissement depuis PHP 8 ; dans d'anciennes versions, cela relevait souvent d'une notice.

```
<?php
// Exemple indépendant, le fichier est volontairement absent.
include "fichier_absent.php";
echo "Le script peut continuer après cet avertissement.";
```

Notice — E_NOTICE et usage obsolète — E_DEPRECATED

Une notice signale un problème moins grave ; son emploi dépend de la version. E_DEPRECATED indique un usage encore accepté mais déconseillé. Depuis PHP 8.1, transmettre null à certains paramètres internes non nullables peut produire ce diagnostic d'obsolescence.

Erreur logique : pas forcément de message PHP

Un calcul faux ou une mauvaise condition peut s'exécuter sans diagnostic. Pour une moyenne, diviser par l'effectif, pas par une constante choisie au hasard. Tester le résultat sur un petit jeu dont on connaît la réponse.

Méthode de recherche

1. Lire le type d'erreur, le nom du fichier et la ligne.
2. Vérifier syntaxe, noms de variables et fichiers nécessaires.
3. Isoler un exemple minimal et comparer résultat attendu et obtenu.
4. Corriger la cause, sans masquer les messages ni exposer des secrets.

Le professeur configure l'affichage des erreurs dans l'environnement local et la journalisation du serveur. Ne pas montrer les diagnostics techniques aux visiteurs d'un site public. La configuration MySQLi peut aussi interrompre un script en cas de requête incorrecte ; les tests de false du cours ne constituent pas une gestion complète de ces erreurs.

SCANNER LA FICHE
ou cliquer sur le QR code

Projet 1 : gestion des élèves — énoncé et base

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Projet adapté de la plateforme

Construire un site « Gestion des élèves — Lycée Rafèha ». La page index.html contient un en-tête, une navigation verticale, un iframe nommé cadre et un pied de page. Les pages d'inscription, de liste, de modification et de statistiques s'affichent dans le cadre.

Base ecole

```
CREATE DATABASE ecole CHARACTER SET utf8mb4;
USE ecole;
CREATE TABLE eleves (
  Id VARCHAR(10) PRIMARY KEY,
  Nom_prenom VARCHAR(60) NOT NULL,
  Moyenne DECIMAL(4,2) NOT NULL CHECK (Moyenne BETWEEN 0 AND 20),
  DateNaissance DATE NOT NULL
);
INSERT INTO eleves VALUES
('A01', 'Benali Amine', 17.50, '2008-03-14'),
('A02', 'Trabelsi Sara', 16.25, '2008-07-02'),
('B01', 'Gharbi Yassine', 14.75, '2007-11-25'),
('C01', 'Ayadi Nour', 10.50, '2008-05-30'),
('D01', 'Cherif Ines', 7.25, '2008-09-17');
```

L'Id commence par une majuscule qui indique sa catégorie. Le nom complet comporte deux à cinq mots de lettres. La moyenne est comprise entre 0 et 20. La date de naissance doit être passée.

Fichiers à produire

index.html, inscription.html, style.css, validation.js, connexion.php, ajouter.php, liste.php, modifier.php et statistiques.php.

Consignes HTML et CSS

Navigation verticale et mise en page avec float, marge latérale et clear. Ne pas employer flex dans ce TP. Les liens ciblent cadre ; l'iframe a un title. Les champs portent label, id et name. Ajouter des transitions sur les liens, boutons et champs au focus.

Consignes JavaScript et PHP

Utiliser let en JavaScript. Nettoyer les espaces, contrôler les champs et afficher les erreurs avec textContent. Refaire tous les contrôles en PHP. Dans la correction simplifiée, distinguer les contrôles des saisies et les requêtes SQL fixes ; seuls des entiers strictement validés peuvent être intégrés aux recherches numériques. La normalisation en majuscules d'un identifiant ne remplace pas sa validation.

Évaluation proposée sur 20

HTML et navigation : 4 ; CSS et transitions : 4 ; validation JavaScript : 4 ; PHP et données : 6 ; tests et lisibilité : 2. Cette répartition est proposée pour cette édition.

Périmètre de la correction PHP

La correction fournie sépare validation des saisies et requêtes à valeurs fixes. L'enregistrement complet du texte libre du formulaire n'est pas réalisé dans cette version pédagogique. Les contrôles métier restent à effectuer côté serveur pour une application complète.

Projet 1 : validation par caractères

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Contrôler un identifiant sans motif régulier

L’identifiant contient une lettre majuscule suivie de lettres majuscules ou de chiffres, pour une longueur totale de 1 à 10. Le formulaire normalise d’abord sa casse.

```
function idValide(id) {
  let lettres = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
  let autorises = lettres + "0123456789";
  if (id.length < 1 || id.length > 10
      || lettres.indexOf(id[0]) === -1) { return false; }
  for (let i = 1; i < id.length; i++) {
    if (autorises.indexOf(id[i]) === -1) { return false; }
  }
  return true;
}
```

Compter et contrôler les mots d’un nom

L’alphabet ci-dessous est une liste pédagogique explicite pour les exercices. Il ne couvre pas tous les noms ni toutes les langues. Les espaces extérieurs sont retirés avant l’appel ; plusieurs espaces intérieurs séparent aussi les mots.

```
function nomValide(nom) {
  let alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZÀÂÄÉÈÊËÎÏÔÖÙÛÜÇŒŸ";
  let mots = 0, dansMot = false;
  if (nom.length > 60) { return false; }
  for (let i = 0; i < nom.length; i++) {
    if (nom[i] === " ") { dansMot = false; }
    else {
      if (alphabet.indexOf(nom[i].toUpperCase()) === -1) { return false; }
      if (!dansMot) { mots++; dansMot = true; }
    }
  }
  return mots >= 2 && mots <= 5;
}
```

Tester indépendamment

idValide("A01") renvoie true ; idValide("1A") renvoie false. nomValide("Ali Benali") renvoie true ; nomValide("Ali") renvoie false. Tester les fonctions séparément avant de les associer au formulaire de la fiche suivante. La correction PHP reste limitée aux requêtes fixes ou à un entier strictement validé.

Projet 1 : formulaire et validation JavaScript

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

inscription.html : contenu du formulaire

```
<form id="inscription" method="post" action="ajouter.php"
      onsubmit="return verifierInscription()">
  <label for="Id">Identifiant</label>
  <input id="Id" name="Id" maxlength="10" required>
  <label for="Nom_prenom">Nom complet</label>
  <input id="Nom_prenom" name="Nom_prenom" maxlength="60" required>
  <label for="Moyenne">Moyenne</label>
  <input id="Moyenne" name="Moyenne" type="number"
         min="0" max="20" step="0.01" required>
  <label for="DateNaissance">Date de naissance</label>
  <input id="DateNaissance" name="DateNaissance" type="date" required>
  <button type="submit">Ajouter</button>
  <button type="reset">Effacer</button>
</form>
<p id="erreurs" role="alert"></p>
<script src="validation.js" defer></script>
```

validation.js

Réutiliser dateNonFuture définie dans la fiche « Médias, styles et dates de formulaire ». Ajouter le contrôle strictement passé indiqué ci-dessous.

```
function verifierInscription() {
  let champId = document.getElementById("Id");
  champId.value = champId.value.trim().toUpperCase();
  let nom = document.getElementById("Nom_prenom").value.trim();
  let brut = document.getElementById("Moyenne").value;
  let note = Number(brut);
  let naissance = document.getElementById("DateNaissance").value;
  let jour = new Date(), m = jour.getMonth() + 1, j = jour.getDate();
  let dateJour = jour.getFullYear() + "-"
    + (m < 10 ? "0" : "") + m + "-" + (j < 10 ? "0" : "") + j;
  let valide = idValide(champId.value) && nomValide(nom)
    && brut.trim() !== "" && !isNaN(note) && note >= 0 && note <= 20
    && dateNonFuture(naissance) && naissance < dateJour;
  document.getElementById("erreurs").textContent =
    valide ? "" : "Vérifier les données.";
  return valide;
}
```

Charger d’abord les fonctions idValide et nomValide de la fiche précédente, ainsi que dateNonFuture du cours. onsubmit transmet le booléen : false bloque l’envoi, true le permet. La comparaison des dates au format AAAA-MM-JJ exclut la date du jour, après validation civile. Refaire les contrôles côté serveur.

Projet 1 : connexion et insertion pédagogique

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

connexion.php — base ecole de l’exercice

```
<?php
$cnx = mysqli_connect("localhost", "utilisateur_cours",
                      "mot_de_passe", "ecole");
if (!$cnx) { die ("Connexion impossible."); }
```

La configuration de cours gère l’encodage et les diagnostics. Le schéma eleves est défini dans l’énoncé. Ce fragment démontre une insertion fixe avant d’aborder le traitement d’un formulaire.

ajouter.php — valeurs fixes, indépendantes de POST

```
<?php
require "connexion.php";
$ok = mysqli_query($cnx,
    "INSERT INTO eleves (Id, Nom_prenom, Moyenne, DateNaissance) "
    . "VALUES ('A01', 'Benali Amine', 15.50, '2009-03-14')"
);
if ($ok === false) { die ("Insertion impossible."); }
echo mysqli_affected_rows($cnx) . " élève ajouté.";
mysqli_close($cnx);
```

L’exemple échoue si A01 existe déjà : restaurer le jeu initial avant de le répéter. Ne pas recopier ce bloc en y injectant directement des valeurs de formulaire.

Valider une moyenne reçue sans l’insérer

```
<?php
if (!isset($_POST["Moyenne"]) || !is_string($_POST["Moyenne"])) {
    die ("Moyenne manquante.");
}
$texte = trim($_POST["Moyenne"]);
if (!is_numeric($texte)) { die ("Moyenne invalide."); }
$moyenne = (float)$texte;
if (!is_finite($moyenne) || $moyenne < 0 || $moyenne > 20) {
    die ("Moyenne hors limites.");
}
echo "Moyenne validée : " . $moyenne;
```

Les tests d’existence, de type et de domaine sont effectués côté serveur. Une date de naissance doit aussi passer checkdate et respecter les bornes métier. Une saisie libre de nom n’est pas insérée dans les scripts simplifiés de ce livre.

Portée de la correction

Le projet conserve son formulaire et sa validation JavaScript. La correction PHP ci-dessus distingue volontairement la validation des entrées et la démonstration SQL à valeurs fixes. Elle ne réalise pas l’enregistrement complet d’un nom libre reçu du navigateur et ne doit pas être présentée comme une application publique sécurisée.

Projet 1 : lecture, modification et statistiques

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

liste.php : n’afficher ici que les résultats numériques

```
<?php
require "connexion.php"; // version ouverte sur ecole
$res = mysqli_query($cnx, "SELECT Moyenne FROM eleves ORDER BY Id");
if ($res === false) { die ("Lecture impossible."); }
echo "<table><tr><th>Moyenne</th></tr>";
while ($e = mysqli_fetch_assoc($res)) {
    echo "<tr><td>" . (float)$e["Moyenne"] . "</td></tr>";
}
echo "</table>";
mysqli_close($cnx);
```

Les noms et identifiants textuels ne sont pas réaffichés librement dans cette version simplifiée. Pour une table vide, la boucle ne produit aucune ligne de données.

modifier.php : requête fixe sur la base de test

```
<?php
require "connexion.php";
$ok = mysqli_query($cnx,
    "UPDATE eleves SET Moyenne = 16.00 WHERE Id = 'A01'"
);
if ($ok === false) { die ("Modification impossible."); }
echo mysqli_affected_rows($cnx) . " ligne(s) modifiée(s).";
mysqli_close($cnx);
```

Zéro peut signifier que A01 n’existe pas ou que la valeur est déjà 16. Ne pas retirer WHERE. Cet exercice est indépendant de l’insertion précédente et doit être testé sur une copie.

Statistiques à exploiter

```
SELECT COUNT(*) AS effectif, AVG(Moyenne) AS moyenne_classe,
       MIN(Moyenne) AS minimum, MAX(Moyenne) AS maximum FROM eleves;
SELECT LEFT(Id, 1) AS categorie, COUNT(*) AS effectif,
       AVG(Moyenne) AS moyenne FROM eleves
GROUP BY LEFT(Id, 1) ORDER BY categorie;
SELECT Id, Nom_prenom,
       TIMESTAMPDIFF(YEAR, DateNaissance, CURRENT_DATE()) AS age
FROM eleves ORDER BY Nom_prenom;
```

Les dernières requêtes sont à consulter dans l’outil SQL de cours ; elles ne justifient pas un affichage HTML brut de texte externe. Prévoir les cas de table vide : un agrégat peut produire NULL. Les anniversaires du jour peuvent être recherchés avec MONTH et DAY.

Projet 2 : recharge électrique — énoncé et base

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Contexte adapté de la plateforme

Gérer les recharges d'une station. Construire la même navigation à iframe que dans le projet précédent, avec float et transitions. Chaque recharge possède Id, Nom_prenom, Borne, Kwh, DateRecharge et HeureRecharge.

La première lettre de l'Id indique le type : L lente, N normale, R rapide, U ultra-rapide. L'énergie est comprise entre 1 et 100 kWh. La date ne doit pas être future ; l'heure doit appartenir aux plages 08:00–12:00 ou 13:00–18:00.

Règle simplifiée retenue pour cette édition

Une borne ne peut recevoir qu'un enregistrement pour une même date et une même heure de départ. Ce modèle ne gère pas encore une durée de recharge ni les chevauchements d'intervalles : ce serait une extension du projet.

```
CREATE DATABASE station CHARACTER SET utf8mb4;
USE station;
CREATE TABLE recharges (
  Id VARCHAR(10) PRIMARY KEY,
  Nom_prenom VARCHAR(60) NOT NULL,
  Borne VARCHAR(10) NOT NULL,
  Kwh DECIMAL(5,2) NOT NULL CHECK (Kwh BETWEEN 1 AND 100),
  DateRecharge DATE NOT NULL,
  HeureRecharge TIME NOT NULL,
  CONSTRAINT uq_depart UNIQUE (Borne, DateRecharge, HeureRecharge),
  CONSTRAINT ck_horaire CHECK (
    HeureRecharge BETWEEN '08:00:00' AND '12:00:00'
    OR HeureRecharge BETWEEN '13:00:00' AND '18:00:00'
  )
);
INSERT INTO recharges VALUES
('L01', 'Benali Amine', 'B01', 20.00, '2026-10-20', '09:00:00'),
('R01', 'Trabelsi Sara', 'B02', 35.50, '2026-10-20', '14:00:00');
```

Livrables

index.html, inscription.html, style.css, validation.js et quatre pages PHP : ajouter, liste, modifier et statistiques. Adapter connexion.php pour utiliser station.

Contrôles et affichage

Nettoyer le nom, contrôler l'Id et l'énergie, vérifier date et heure en JavaScript puis PHP. Afficher une horloge et changer la couleur de fond parmi trois couleurs prédéfinies. Fournir un tableau de l'activité du jour, sans présenter une borne comme « libre maintenant » sur la seule base d'une heure de départ.

Périmètre de la correction PHP

La correction fournie sépare validation des saisies et requêtes à valeurs fixes. L'enregistrement complet du texte libre du formulaire n'est pas réalisé dans cette version pédagogique. Les contrôles métier restent à effectuer côté serveur pour une application complète.

Projet 2 : horaires et insertion pédagogique

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Validation JavaScript d’un horaire

```
function horaireValide(heure) {
  if (heure.length !== 5 || heure[2] !== ":") { return false; }
  for (let i = 0; i < 5; i++) {
    if (i !== 2 && (heure[i] < "0" || heure[i] > "9")) { return false; }
  }
  let h = Number(heure.substring(0, 2));
  let m = Number(heure.substring(3, 5));
  if (h > 23 || m > 59) { return false; }
  let minutes = h * 60 + m;
  return (minutes >= 480 && minutes <= 720)
      || (minutes >= 780 && minutes <= 1080);
}
```

Cette édition inclut les bornes 12:00 et 18:00 comme heures de départ autorisées. Modifier explicitement la règle si une durée minimale doit être terminée avant la fermeture.

Validation PHP d’un horaire reçu

```
<?php
if (!isset($_POST["HeureRecharge"]) || !is_string($_POST["HeureRecharge"])) {
    die ("Heure manquante.");
}
$heure = $_POST["HeureRecharge"];
if (strlen($heure) !== 5 || $heure[2] !== ":"
    || !is_numeric (substr($heure, 0, 2))
    || !is_numeric (substr($heure, 3, 2))) {
    die ("Heure invalide.");
}
$h = (int)substr($heure, 0, 2); $m = (int)substr($heure, 3, 2);
$minutes = $h * 60 + $m;
if ($h > 23 || $m > 59 || !(($minutes >= 480 && $minutes <= 720)
    || ($minutes >= 780 && $minutes <= 1080))) {
    die ("Hors horaires.");
}
echo "Horaire valide.";
```

Insertion fixe indépendante dans station

```
<?php
// $cnx ouverte sur station ; identifiants R02 et B02 de démonstration.
$ok = mysqli_query($cnx,
    "INSERT INTO recharges "
    . "(Id, Nom_prenom, Borne, Kwh, DateRecharge, HeureRecharge) "
    . "VALUES ('R02', 'Benali Amine', 'B02', 25.00, '2026-10-20', '15:00:00')"
);
if ($ok === false) { die ("Insertion impossible."); }
echo mysqli_affected_rows($cnx) . " recharge ajoutée.";
mysqli_close($cnx);
```

Cette insertion ne reprend aucune saisie libre du formulaire. Réinitialiser le jeu avant de répéter. La contrainte UNIQUE empêche deux départs à la même heure sur la même borne ; elle ne gère pas les durées ni les chevauchements.

Projet 2 : statistiques, coûts et tests

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Statistiques SQL

```
SELECT LEFT(Id, 1) AS type_recharge,
       COUNT(*) AS nombre, SUM(Kwh) AS energie
FROM recharges GROUP BY LEFT(Id, 1) ORDER BY type_recharge;
SELECT Borne, HeureRecharge, Id, Kwh
FROM recharges WHERE DateRecharge = CURRENT_DATE()
ORDER BY Borne, HeureRecharge;
```

Le second résultat est un planning des départs enregistrés, pas une preuve de disponibilité en temps réel.

Calcul d'un coût estimé en PHP

Les tarifs suivants sont inventés pour l'exercice et exprimés en DT/kWh ; ils ne représentent pas une grille commerciale réelle.

```
<?php
// $cnx ouverte sur station ; seules des sorties numériques sont produites.
$tarifs = ["L" => 0.400, "N" => 0.500, "R" => 0.650, "U" => 0.800];
$res = mysqli_query($cnx, "SELECT Id, Kwh FROM recharges ORDER BY Id");
if ($res === false) { die ("Lecture impossible."); }
while ($ligne = mysqli_fetch_assoc($res)) {
    $type = substr($ligne["Id"], 0, 1);
    if (!isset($tarifs[$type])) { continue; }
    $cout = (float)$ligne["Kwh"] * $tarifs[$type];
    echo round($cout, 3) . " DT<br>";
}
mysqli_close($cnx);
```

Pour une facturation réelle, enregistrer le tarif appliqué, fixer une règle d'arrondi et employer des calculs décimaux adaptés. Ne pas recalculer un ancien montant avec un tarif qui a changé.

Jeu de tests obligatoire

- Énergie : 0, 1, 100, 101 ; nom vide ou contenant un chiffre.
- Horaires : 07:59, 08:00, 12:00, 12:01, 13:00, 18:00, 18:01.
- Date future et date civile impossible ; doublon d'Id ; doublon de départ sur la même borne.
- Texte libre : ne pas le concaténer dans SQL ni le réafficher brut ; base vide ; serveur indisponible.
- Validation contournée : soumettre directement une requête POST invalide et vérifier le refus PHP.

Travail complémentaire

Ajouter l'heure de fin et empêcher les chevauchements pour une borne. Définir une stratégie transactionnelle avant de prétendre gérer les réservations simultanées. Prévoir autorisations et jeton CSRF pour toute modification sensible.

Méthode de résolution et lecture de programme

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Une démarche en six étapes

1. Analyser : souligner l'action demandée et les contraintes ; reformuler le résultat à obtenir.
2. Repérer : distinguer les données d'entrée, les résultats de sortie et les règles de validité.
3. Préparer : choisir les éléments HTML, les styles, les variables et les traitements nécessaires.
4. Décomposer : écrire les étapes en français ou en pseudo-code avant de programmer.
5. Réaliser : construire une petite partie à la fois et vérifier chaque résultat intermédiaire.
6. Tester : comparer le résultat attendu au résultat obtenu, puis corriger et tester à nouveau.

Suivre le cycle d'une application

Navigateur → formulaire → requête HTTP → script PHP → base MySQL → réponse HTML → navigateur. Le navigateur n'accède pas directement à la base : PHP effectue les traitements côté serveur.

Répartir les responsabilités

HTML définit les champs ; CSS présente la page ; JavaScript aide à la saisie. PHP vérifie à nouveau les données et décide du traitement. SQL interroge ou modifie la base. Une validation JavaScript réussie ne garantit pas la validité des données reçues par le serveur.

Lire un programme : suivre les variables

```
let notes = [8, 12, 15];
let total = 0;
for (let i = 0; i < notes.length; i++) {
  total += notes[i];
}
```

Passage	Valeur ajoutée	Total avant	Total après
i = 0	notes[0] = 8	0	8
i = 1	notes[1] = 12	8	20
i = 2	notes[2] = 15	20	35
Fin : i = 3	Condition fausse	35	35

Après le dernier passage, i vaut 3 : i < notes.length est faux. La boucle s'arrête ; total vaut 35. Écrire ce tableau de trace permet de vérifier l'initialisation, les indices et le cumul.

Erreurs fréquentes et réflexes de correction

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Localiser avant de modifier

Reproduire le problème et identifier la couche concernée. Lire le message éventuel, relever le fichier et la ligne, puis vérifier aussi les lignes précédentes. Corriger une cause à la fois : plusieurs modifications simultanées rendent la vérification difficile.

Domaine	Erreur fréquente	Réflexe de correction
HTML	Balises mal imbriquées	Vérifier les ouvertures, fermetures et l'ordre des éléments.
HTML	Même id pour deux éléments	Donner un id unique à chaque élément du document.
HTML	Champ transmis sans name	Distinguer id (repérage) et name (nom de la donnée envoyée).
CSS	Sélecteur incorrect ou fichier non chargé	Vérifier le chemin de la feuille, puis la classe ou l'identifiant visé.
CSS	Règle sans effet	Examiner la syntaxe, la cascade et les styles calculés dans le navigateur.
JavaScript	Confusion entre = et ===	= affecte une valeur ; === compare sans conversion de type.
JavaScript	Calcul sur un champ au lieu de sa valeur	Lire .value ; vérifier la chaîne vide, convertir avec Number et contrôler les bornes.
JavaScript	Indice de tableau hors limites	Pour un tableau dense, parcourir les indices de 0 à length - 1.
JavaScript	Élément non trouvé	Vérifier l'id et attendre le chargement du document, par exemple avec defer.
SQL	Condition WHERE oubliée	Avant UPDATE ou DELETE, relire la condition et vérifier les lignes visées sur une copie.
PHP	Donnée reçue supposée valide	Vérifier présence, type, format et règles métier côté serveur.
PHP / SQL	Résultat vide confondu avec un échec	Une sélection réussie peut ne retourner aucune ligne ; distinguer ce cas d'un échec.

Une erreur peut rester silencieuse

Une moyenne calculée avec un mauvais diviseur ou un filtre SQL incorrect peut donner un résultat plausible sans message d'erreur. Comparer avec un petit jeu de données dont le résultat est connu.

Une correction doit être vérifiée

Après la correction, rejouer le cas qui échouait puis un cas qui fonctionnait. Dans le navigateur, la console et l'inspection des éléments aident à localiser la cause ; elles ne remplacent pas les tests du résultat.

Lire les diagnostics PHP

Utiliser les deux fiches consacrées aux erreurs PHP pour distinguer syntaxe, erreur fatale, avertissement et obsolescence. Les messages dépendent de la version ; ne pas exposer les détails techniques aux visiteurs d'un site public.

SCANNER LA FICHE
ou cliquer sur le QR code

Tests, vérification et checklist de remise

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Construire un petit plan de vérification

Un test décrit une entrée ou une action, le résultat attendu et le résultat obtenu. Décider de l'attendu avant d'exécuter le programme. Noter ensuite « conforme » ou décrire précisément l'écart observé.

Exemple : note obligatoire comprise entre 0 et 20

Le zéro est une valeur valide : il ne faut pas le confondre avec un champ vide. Vérifier le texte saisi avant la conversion et les bornes après la conversion. Le tableau suivant sert de modèle de compte rendu, pas de nouvelle série d'exercices.

Entrée	Résultat attendu	Résultat obtenu
Chaîne vide ou espaces	Saisie refusée	À relever
Note : 0	Saisie acceptée	À relever
Note : 20	Saisie acceptée	À relever
Note : -1 ou 21	Saisie refusée	À relever
Texte non numérique	Saisie refusée	À relever

Adapter les cas au traitement

• Valeur normale : une donnée représentative, par exemple une note de 12.

• Valeurs limites : 0 et 20, puis une valeur juste en dessous ou au-dessus si les décimales sont autorisées.

• Cas absents ou incorrects : champ vide, espaces, format incompatible.

• Collections : aucun élément, un seul élément, plusieurs éléments.

• Sélection SQL : aucune ligne, une seule ligne, plusieurs lignes ; vérifier le filtre et l'ordre.

Vérifier sans risquer les données

Tester les modifications de base sur une copie sauvegardée. Avant une opération destructive, vérifier les lignes visées et relire la condition. Ne pas répéter une insertion sans tenir compte des contraintes d'unicité.

Checklist avant de rendre le travail

☐ Les fichiers existent, les chemins sont corrects et les ressources sont chargées.

☐ Les noms sont cohérents ; les champs ont leurs libellés et les attributs nécessaires.

☐ Les traitements produisent les résultats attendus, y compris aux limites.

☐ Les erreurs sont compréhensibles ; une correction n'a pas cassé un cas déjà valide.

☐ Le code est lisible et les résultats de vérification sont notés.

Vérification particulière du serveur

Refaire les contrôles côté PHP, même si le formulaire semble valide. Distinguer résultat SQL vide et échec de requête ; la configuration MySQLi peut interrompre un script. Les exemples à requêtes fixes ou entiers validés du livre restent pédagogiques, pas une sécurité de production complète.

Synthèse et contrôle des acquis

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Parcours d’un projet Web

1. Définir les données et les règles de gestion.

2. Concevoir le schéma relationnel et ses contraintes.

3. Construire une page HTML sémantique et des formulaires accessibles.

4. Appliquer les styles CSS, contrôler le modèle de boîte et la lisibilité.

5. Ajouter JavaScript avec let, des fonctions et des écouteurs d’événements.

6. Valider les données côté PHP, utiliser les fonctions MySQLi procédurales retenues et des requêtes fixes ou à entier validé.

7. Échapper les sorties selon leur contexte et tester les erreurs.

Vérification technique

• HTML : id uniques, name sur les champs à transmettre, label correctement associés.

• CSS : unités sans espace, bordures incluses dans le calcul des dimensions.

• JavaScript : déclarations avant utilisation, portée de bloc, gestion de null et de NaN.

• SQL : schéma sélectionné, noms de colonnes cohérents, clés étrangères explicites, WHERE vérifié avant UPDATE/DELETE.

• PHP : balise <?php, tableaux parcourus avec for ou while, aucune ancienne API de connexion, contrôle des valeurs reçues.

• Dates : fuseau configuré, formats Y-m-d et H:i:s, timestamps en secondes en PHP et en millisecondes en JavaScript.

Exercices de synthèse

A. Réaliser un formulaire d’inscription d’élèves et afficher les données reçues sans injection HTML.

B. Afficher les locations d’un client pour une période, avec une requête fixe à adapter dans l’environnement de cours.

C. Calculer une échéance à sept jours puis vérifier que l’affichage est au format français.

D. Dans commerce, lister les articles jamais vendus et les clients dont le total dépasse 1000 DT.

Repères documentaires officiels

HTML et CSS : https://developer.mozilla.org/fr/

JavaScript : https://developer.mozilla.org/fr/docs/Web/JavaScript

PHP et MySQLi : https://www.php.net/manual/fr/

MySQL 8.0 : https://dev.mysql.com/doc/refman/8.0/en/

Révision : questions et réponses essentielles

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Questions

1. Quelle différence entre id, class et name en HTML ?

2. Un style interne est-il toujours prioritaire sur un style externe ?

3. Quelle place conserve un élément positionné en relative ?

4. Pourquoi i n’est-il plus accessible après for (let i = 0; ...) ?

5. Pourquoi tester prompt contre null avant conversion ?

6. Pourquoi COUNT(*) et COUNT(colonne) peuvent-ils différer ?

7. Où placer un filtre portant sur un agrégat de groupes ?

8. Pourquoi une requête SELECT COUNT(*) retourne-t-elle une ligne ?

9. checkdate renvoie-t-elle un timestamp ?

10. Comment protéger les valeurs SQL et les sorties HTML ?

Réponses

1. id identifie un élément unique ; class sert à partager des sélecteurs ; name est notamment la clé de transmission d’un champ de formulaire.

2. Non. L’emplacement interne/externe ne suffit pas ; considérer origine, importance, couches, spécificité et ordre d’apparition.

3. Sa place initiale reste réservée, même si son rendu est décalé.

4. let déclare la variable dans la portée de la boucle, et non à l’extérieur.

5. Annuler renvoie null ; Number(null) vaut 0 et pourrait transformer une annulation en saisie numérique valide.

6. COUNT(colonne) ignore NULL ; COUNT(*) compte les lignes.

7. Dans HAVING après GROUP BY. WHERE filtre les lignes avant regroupement.

8. Sans GROUP BY, l’agrégation globale produit une ligne contenant le compteur, même si la table est vide ; mysqli_num_rows vaut alors 1.

9. Non : elle renvoie un booléen et vérifie une date civile dans l’ordre mois, jour, année.

10. Distinguer validation, requête SQL et affichage HTML. Les exemples simplifiés utilisent des données fixes ou numériques contrôlées ; ne pas y introduire du texte libre reçu.

Auto-évaluation

Pour chaque réponse, produire un exemple de cinq à dix lignes. Tester un cas normal, un cas limite et un cas invalide. Les tests de sécurité ne doivent être réalisés que sur votre propre environnement pédagogique.

Sources, périmètre et conventions de cette édition

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Supports utilisés

• PDF original fourni : Cours html5-js-PHP-BD -.pdf, 65 pages.

• Application pédagogique : STI V2.0 BY A'e

• Parties consultées : HTML5, CSS3, JavaScript, SQL, PHP et MySQLi. Accès par les QR codes du manuel.

Édition Smart Book

Cette édition connectée associe des liens et QR codes aux fiches. Un filigrane QR très pâle renvoie à la plateforme ; il sert de marquage, pas de code de lecture fiable ni de protection contre la copie. Le code contrasté en bas de page est celui à scanner. Cette édition fusionne la première réorganisation du PDF avec les compléments des pages de cours de la plateforme. Les explications et exemples sont reformulés pour une lecture imprimée. Elle ne reproduit pas toutes les animations, vidéos, fenêtres interactives, quiz et corrections téléchargeables. Les deux projets de synthèse sont des adaptations pédagogiques avec fragments de correction ; les tarifs et certaines règles simplifiées y sont signalés comme choix de cette édition.

Conventions techniques

• JavaScript : let pour les variables et gestion explicite de leur portée.

• PHP : syntaxe PHP 8, tableaux parcourus avec for ou while, sans ancienne fonction de parcours supprimée.

• Accès aux bases : uniquement MySQLi procédural, appels mysqli_* ; aucune API mysql_* ni accès orienté objet ou PDO.

• SQL : exemples MySQL 8.0.16 ou ultérieur ; contraintes référentielles explicites et attention au moteur et à la version.

• Dates : exemples actualisés pour 2026 ; 2028 pour illustrer une année bissextile ; fuseau PHP Africa/Tunis.

Documentation de référence

HTML, CSS et JavaScript : https://developer.mozilla.org/fr/

PHP : https://www.php.net/manual/fr/

MySQL : https://dev.mysql.com/doc/refman/8.0/en/

Mise en œuvre

Les schémas gestion_location, commerce, ecole et station sont des domaines distincts. Sélectionner la bonne base avant chaque activité. Certains fragments supposent des variables validées ou une connexion déjà ouverte, explicitement indiquées dans le texte. Les opérations destructives et les changements de structure doivent être testés sur une copie sauvegardée.

Pour une mise en production

Les projets de lycée ne constituent pas une application publique sécurisée complète. Ajouter notamment authentification, autorisations, protection CSRF, configuration des secrets, journalisation et stratégie de concurrence avant un déploiement réel.

Complément utilisé

Support PHP-recap.html fourni par l’enseignant : include/require, isset/empty, GET/POST, calculs, chaînes, conversions, guillemets, formats de date et cycle MySQLi. Les fonctionnalités non retenues dans la liste du cours sont exclues des exemples.

HTML5 — balises et attributs

À gauche : la balise ; au centre : son rôle ; à droite : les attributs rencontrés ou explicitement étudiés dans le livre. Les attributs globaux tels que id, class, style, title, lang et data-* peuvent être utilisés sur les éléments appropriés. Ce tableau n'est pas un catalogue de tous les attributs HTML.

Balise	Rôle	Attributs à droite
<html>	Racine du document	lang
<head>	Métadonnées du document	Attributs globaux
<body>	Contenu de la page	Attributs globaux
<title>	Titre de l'onglet	Attributs globaux
<meta>	Métadonnée	charset, content, name
<link>	Ressource liée	href, media, rel
<style>	Feuille CSS intégrée	media
<script>	Script intégré ou externe	defer, src, type
<header>	En-tête de page ou de section	Attributs globaux
<nav>	Zone de navigation	aria-label
<main>	Contenu principal	Attributs globaux
<section>	Section thématique	class
<article>	Contenu autonome	class, data-code, lang
<aside>	Contenu complémentaire	Attributs globaux
<footer>	Pied de page ou de section	Attributs globaux
	Conteneur de bloc générique	class
	Conteneur en ligne générique	aria-label, class, id
<p>	Paragraphe	class, id, role, style, title
<h1>	Titre de niveau 1	class, id
<h2>	Titre de niveau 2	id
<h3>	Titre de niveau 3	Attributs globaux
<h4>	Titre de niveau 4	Attributs globaux
<h5>	Titre de niveau 5	Attributs globaux

Balise	Rôle	Attributs à droite
<h6>	Titre de niveau 6	Attributs globaux
 	Saut de ligne	Attributs globaux
<hr>	Séparation thématique	Attributs globaux
	Importance forte	Attributs globaux
	Emphase	Attributs globaux
<mark>	Mise en évidence	Attributs globaux
<abbr>	Abréviation	title
<sub>	Indice	Attributs globaux
<sup>	Exposant	Attributs globaux
<blockquote>	Citation en bloc	cite
<q>	Citation courte	cite
<cite>	Titre d’une œuvre	Attributs globaux
<del>	Texte supprimé	cite, datetime
<ins>	Texte ajouté	cite, datetime
<pre>	Texte préformaté	Attributs globaux
<code>	Fragment de code	Attributs globaux
<address>	Coordonnées de contact	Attributs globaux
<time>	Date ou heure	datetime
<a>	Lien hypertexte	href, rel, target
<img>	Image	alt, class, height, id, src, width
<figure>	Illustration autonome	Attributs globaux
<figcaption>	Légende d’une figure	Attributs globaux
<audio>	Lecture audio	controls, src
<video>	Lecture vidéo	controls, height, id, src, width
<source>	Source de média	media, src, type
<iframe>	Document intégré	height, id, name, sandbox, src, title, width

Balise	Rôle	Attributs à droite
	Liste non ordonnée	Attributs globaux
	Liste ordonnée	type
	Élément de liste	value
<dl>	Liste de descriptions	Attributs globaux
<dt>	Terme à définir	Attributs globaux
<dd>	Description du terme	Attributs globaux
<table>	Tableau de données	Attributs globaux
<caption>	Titre du tableau	Attributs globaux
<thead>	Groupe de lignes d’en-tête	Attributs globaux
<tbody>	Corps du tableau	Attributs globaux
<tfoot>	Pied du tableau	Attributs globaux
<tr>	Ligne du tableau	Attributs globaux
<th>	Cellule d’en-tête	colspan, rowspan, scope
<td>	Cellule de données	colspan, rowspan
<form>	Formulaire	action, enctype, id, method, onsubmit, target
<fieldset>	Groupe de champs	disabled
<legend>	Légende du groupe	Attributs globaux
<label>	Libellé associé à un champ	for
<input>	Champ de saisie	checked, disabled, id, list, max, maxlength, min, multiple, name, pattern, placeholder, readonly, required, step, type, value
<select>	Liste de choix	disabled, id, multiple, name, required, size
<option>	Choix d’une liste	disabled, selected, value
<optgroup>	Groupe de choix	disabled, label
<textarea>	Saisie multiligne	disabled, id, maxlength, name, placeholder, readonly, required, rows
<button>	Bouton	disabled, id, name, onclick, type, value
<datalist>	Suggestions de saisie	id



<!DOCTYPE html> est une déclaration de type de document, pas une balise. Les gestionnaires on… rencontrés dans les exemples sont des attributs d’événement ; privilégier les écouteurs JavaScript quand le cours le demande.

SCANNER OU CLIQUER



CSS3 — aide-mémoire par thèmes

Les sélecteurs sont présentés par familles ; les propriétés sont regroupées par rôle. Les valeurs données sont des exemples à adapter. Une unité s'écrit sans espace, par exemple 12px.

Sélecteurs

Élément	Syntaxe / exemple	Description
balise	p { … }	Sélectionner tous les éléments de ce type.
classe	.important { … }	Sélectionner une classe réutilisable.
identifiant	#intro { … }	Sélectionner l'élément portant cet identifiant unique.
universel	* { … }	Sélectionner tous les éléments.
réunion	h1, h2 { … }	Appliquer une règle à plusieurs sélecteurs.
descendant	nav a { … }	Sélectionner les descendants correspondants.
enfant direct	nav > a { … }	Limiter aux enfants directs.
classes combinées	.carte.titre { … }	Exiger les deux classes sur le même élément.
attribut	input[type="email"] { … }	Sélectionner selon un attribut.
pseudo-classes	a:hover, input:focus { … }	Sélectionner un état ou une position.
position	li:first-child, tr:nth-child(even)	Cibler le premier enfant ou les lignes paires.
focus visible	:focus-visible { … }	Mettre en évidence la navigation au clavier.

Texte et police

Élément	Syntaxe / exemple	Description
font-family	font-family: Arial, sans-serif;	Définir une famille de police et un repli.
font-size	font-size: 18px;	Régler la taille de police.
font-weight	font-weight: bold;	Régler la graisse.
font-style	font-style: italic;	Régler le style de police.
line-height	line-height: 1.5;	Régler la hauteur de ligne.
text-align	text-align: center;	Aligner le contenu en ligne.
text-decoration	text-decoration: underline;	Ajouter ou retirer une décoration.
text-shadow	text-shadow: 1px 1px 2px gray;	Créer une ombre sur le texte.

Complément — variantes utilisées dans les fiches
transform : translateX(px) · translateY(px) · skewX(deg) · skewY(deg)
Fonds : linear-gradient(direction, couleur1, couleur2, …) · Couleurs : rgb(r, v, b) · hsl(h, s%, l%)
Filtres : blur(px) · invert(%)

Élément	Syntaxe / exemple	Description
overflow-wrap	overflow-wrap: anywhere;	Autoriser la coupure d’un mot trop long.

Arrière-plan et couleur

Élément	Syntaxe / exemple	Description
color	color: navy;	Définir la couleur du texte.
background	background: #eef4f7;	Définir les composantes de fond en raccourci.
background-color	background-color: white;	Définir la couleur de fond.
background-size	background-size: cover;	Dimensionner l’image de fond.
background-repeat	background-repeat: no-repeat;	Contrôler la répétition du fond.
background-position	background-position: center;	Positionner le fond.

Boîte, dimensions et bordures

Élément	Syntaxe / exemple	Description
width	width: 300px;	Régler la largeur.
height	height: 200px;	Régler la hauteur.
min-height	min-height: 100px;	Fixer une hauteur minimale.
max-height	max-height: 300px;	Fixer une hauteur maximale.
box-sizing	box-sizing: border-box;	Inclure bordures et padding dans les dimensions.
margin	margin: 12px;	Créer une marge extérieure.
margin-left	margin-left: 10px;	Régler la marge gauche.
margin-right	margin-right: 10px;	Régler la marge droite.
padding	padding: 12px;	Créer un espacement intérieur.
padding-left	padding-left: 20px;	Régler l’espacement intérieur gauche.
border	border: 1px solid gray;	Définir épaisseur, style et couleur de bordure.
border-left	border-left: 3px solid teal;	Définir la bordure gauche.
border-radius	border-radius: 8px;	Arrondir les coins.
outline	outline: 2px solid teal;	Dessiner un contour sans modifier la boîte.

Élément	Syntaxe / exemple	Description
box-shadow	box-shadow: 0 2px 6px gray;	Ajouter une ombre à la boîte.

Disposition et positionnement

Élément	Syntaxe / exemple	Description
display	display: block;	Choisir le mode de disposition.
float	float: left;	Faire flotter un élément.
clear	clear: both;	Se placer après les flottants concernés.
position	position: relative;	Choisir le mode de positionnement.
top	top: 10px;	Décaler depuis le haut, selon position.
right	right: 10px;	Décaler depuis la droite.
bottom	bottom: 10px;	Décaler depuis le bas.
left	left: 10px;	Décaler depuis la gauche.
z-index	z-index: 2;	Régler l’empilement dans son contexte.
overflow	overflow: auto;	Gérer le contenu qui déborde.

Flexbox

Élément	Syntaxe / exemple	Description
flex-direction	flex-direction: row;	Définir l’axe principal.
flex-wrap	flex-wrap: wrap;	Permettre le retour à la ligne.
justify-content	justify-content: center;	Aligner selon l’axe principal.
align-items	align-items: center;	Aligner selon l’axe transversal.
gap	gap: 12px;	Espacer les éléments.
flex	flex: 1;	Régler la flexibilité d’un élément.

Listes et tableaux

Élément	Syntaxe / exemple	Description
list-style	list-style: square inside;	Définir marqueur et position en raccourci.
list-style-type	list-style-type: square;	Choisir le type de marqueur.
list-style-image	list-style-image: url("puce.png");	Choisir une image de marqueur.

SCANNER OU CLIQUER

Élément	Syntaxe / exemple	Description
list-style-position	list-style-position: inside;	Placer le marqueur dedans ou dehors.
border-collapse	border-collapse: collapse;	Fusionner les bordures du tableau.

Effets et transformations

Élément	Syntaxe / exemple	Description
opacity	opacity: 0.8;	Régler l’opacité de l’élément et de son contenu.
filter	filter: grayscale(1);	Appliquer un filtre visuel.
transform	transform: rotate(5deg);	Appliquer translation, rotation, échelle ou inclinaison.
transform-origin	transform-origin: center;	Choisir le point de référence de la transformation.

Transitions et animations

Élément	Syntaxe / exemple	Description
transition	transition: opacity 0.3s ease;	Définir une transition en raccourci.
animation	animation: apparition 1s ease;	Définir une animation en raccourci.
animation-iteration-count	animation-iteration-count: 2;	Choisir le nombre de cycles.
@keyframes	@keyframes nom { from { … } to { … } }	Définir les étapes d’une animation.

Accessibilité et adaptation

Élément	Syntaxe / exemple	Description
@media	@media (prefers-reduced-motion: reduce) { … }	Adapter les styles, notamment pour réduire les mouvements.

JavaScript — syntaxe et API utilisées

Les méthodes appelées sur des variables différentes sont regroupées sous objet.methode(…). Les noms Math, Number, document et console désignent des objets ou fonctions intégrés ; les fonctions personnalisées restent identifiées séparément.

Structure / expression	Rôle
let nom = valeur;	Déclarer une variable de portée de bloc.
if (condition) { … } else { … }	Choisir entre deux traitements.
switch (valeur) { case x: …; break; default: … }	Choisir selon une valeur.
for (let i = 0; i < t.length; i++) { … }	Parcourir un tableau par indices.
while (condition) { … } / do { … } while (condition);	Répéter selon une condition.
for (let valeur of tableau) { … }	Parcourir les valeurs ; syntaxe distincte d'un appel de méthode.
break; / continue;	Quitter une boucle / passer à l'itération suivante.
function nom(parametres) { return resultat; }	Définir une fonction et renvoyer un résultat.
t[t.length] = valeur;	Ajouter une valeur à la fin par son indice.
t[i] / t.length	Lire une case / la longueur du tableau.
chaine[indice]	Lire l'unité de code à cet indice ; undefined si l'indice est hors limites.
= / === / !== / && / || / !	Affecter, comparer strictement, combiner ou nier des conditions.

Fonctions, constructeurs et méthodes

Syntaxe abrégée	Description
objet.addEventListener(…)	Enregistrer un écouteur d'événement.
objet.catch(…)	Traiter le rejet d'une promesse, notamment play().
objet.concat(…)	Concaténer sans modifier l'original.
objet.toString(…)	Convertir en chaîne ; un entier peut recevoir une base.
objet.toUpperCase(…)	Convertir en majuscules.
objet.trim(…)	Retirer les blancs aux extrémités.
Date(…)	Construire une date ; new Date(annee, moisIndex, jour), mois de 0 à 11.
Math.abs(…)	Valeur absolue.

Complément — autres API utilisées dans les fiches
Conversions : parseInt(ch) · parseFloat(ch) · x.valueOf() · Dialogues : confirm(message) → true / false
Minuteurs : setTimeout(fonction, délai) · clearInterval(id) · removeEventListener(événement, fonction)
Dates : d.getTime() · d.setTime(ms)

Syntaxe abrégée	Description
Math.ceil(…)	Entier supérieur ou égal.
Math.floor(…)	Entier inférieur ou égal.
Math.max(…)	Maximum.
Math.min(…)	Minimum.
Math.pow(…)	Puissance.
Math.random(…)	Nombre pseudo-aléatoire dans [0,1[.
Math.round(…)	Arrondi à l'entier le plus proche.
Math.sqrt(…)	Racine carrée.
Math.trunc(…)	Retirer la partie fractionnaire.
Number(…)	Convertir en nombre ; contrôler NaN et les bornes.
Number.isInteger(…)	Tester un entier.
Number.isNaN(…)	Tester NaN sans conversion.
afficher()	Fonction définie dans le cours : lire sa déclaration et ses paramètres dans la fiche correspondante.
afficherHorloge()	Fonction définie dans le cours : lire sa déclaration et ses paramètres dans la fiche correspondante.
alert(…)	Afficher une boîte de message.
objet.indexOf(…)	Indice de la première occurrence ; -1 si absente.
objet.add(…)	Ajouter une classe à classList.
console.log(…)	Écrire dans la console de développement.
objet.getDate(…)	Jour du mois local, de 1 à 31.
objet.getFullYear(…)	Année complète.
objet.getHours(…)	Heures locales.
objet.getMinutes(…)	Minutes locales.
objet.getMonth(…)	Mois local, de 0 à 11.
objet.getSeconds(…)	Secondes locales.
objet.setFullYear(…)	Définir l'année, éventuellement le mois (0 à 11) et le jour.
objet.setHours(…)	Modifier heure, minutes, secondes et millisecondes.

Syntaxe abrégée	Description
dateNonFuture(valeur)	Fonction définie dans le cours : lire sa déclaration et ses paramètres dans la fiche correspondante.
objet.getElementById(…)	Sélectionner par identifiant.
objet.getElementsByClassName(…)	Collection des éléments portant une classe.
objet.getElementsByName(…)	Collection des éléments portant un attribut name donné.
objet.getElementsByTagName(…)	Collection des éléments portant un nom de balise.
doubler(nombre)	Fonction définie dans le cours : lire sa déclaration et ses paramètres dans la fiche correspondante.
objet.pause(…)	Mettre un média en pause.
objet.play(…)	Démarrer un média ; renvoie une promesse.
objet.substring(…)	Extraire une sous-chaîne ; fin exclue.
horaireValide(heure)	Fonction définie dans le cours : lire sa déclaration et ses paramètres dans la fiche correspondante.
idValide(id)	Fonction définie dans le cours : lire sa déclaration et ses paramètres dans la fiche correspondante.
isNaN(…)	Tester NaN après conversion préalable.
nomValide(nom)	Fonction définie dans le cours : lire sa déclaration et ses paramètres dans la fiche correspondante.
objet.sort(…)	Trier le tableau ; fournir (a,b) => a-b pour des nombres.
objet.setAttribute(…)	Définir un attribut HTML.
prompt(…)	Demander une chaîne ; null si annulation.
objet.lastIndexOf(…)	Indice de la dernière occurrence ; -1 si absente.
objet.slice(…)	Extraire sans modifier l’original ; fin exclue.
objet.toLowerCase(…)	Convertir en minuscules.
reagir(event)	Fonction définie dans le cours : lire sa déclaration et ses paramètres dans la fiche correspondante.
setInterval(…)	Répéter une fonction selon un délai.
validerNote()	Fonction définie dans le cours : lire sa déclaration et ses paramètres dans la fiche correspondante.
verifierInscription()	Fonction définie dans le cours : lire sa déclaration et ses paramètres dans la fiche correspondante.

Contre-exemples barrés — ne pas utiliser

Instruction exclue	Alternative du cours
~~Number.isFinite(...)~~	Non retenu : convertir puis contrôler isNaN et les bornes du domaine.
~~Option(...)~~	Non retenu : écrire les options directement dans le HTML.

Propriétés et collections

Propriété	Rôle
length	Longueur d’une chaîne ou d’un tableau.
value	Valeur d’un champ de formulaire.
checked	État coché d’un champ.
selectedIndex	Indice de l’option sélectionnée.
options	Collection des options d’une liste.
textContent	Texte du contenu, sans interprétation HTML.
innerHTML	Contenu interprété comme HTML ; pas de saisie libre non protégée.
style	Styles en ligne de l’élément.
className	Chaîne de classes ; une affectation remplace les classes existantes.
classList	Liste des classes CSS.
src	Adresse d’une ressource.
disabled	État désactivé d’un champ ou bouton.

Bases de données et SQL — synthèse

Notion	Définition / repère
Base de données	Ensemble organisé et persistant de données relatives à un domaine.
SGBD	Logiciel qui définit, manipule, contrôle et protège les données ; MySQL est relationnel.
Table / relation	Ensemble de lignes décrites par les mêmes colonnes.
Champ / colonne / attribut	Propriété portant un nom, un type et éventuellement des contraintes.
Ligne / enregistrement / n-uplet	Occurrence décrite par les valeurs des champs.
Clé primaire (PK)	Champ ou groupe de champs unique et non nul identifiant une ligne ; clés soulignées dans les schémas.
Clé étrangère (FK)	Champ(s) référençant une clé de la table liée ; # est une convention du schéma, pas du code SQL.
Clé composée	Plusieurs champs forment ensemble une seule clé.
Relation 1:1	Une occurrence correspond au plus à une autre ; préciser si les participations sont obligatoires.
Relation 1:N	Une occurrence peut correspondre à plusieurs autres ; la clé étrangère se place du côté N.
Relation N:M	Plusieurs occurrences des deux côtés ; utiliser une table d'association portant les références.
Cardinalités	(0,1), (1,1), (0,N), (1,N) indiquent minimum et maximum de participation.
NULL	Information absente ou inconnue ; différent de zéro et d'une chaîne vide.

Exemple de notation : CLASSE(id_classe, nom_classe) ; ELEVE(num_inscription, nom, id_classe#). Les champs soulignés sont des clés primaires. Dans PARTICIPATION(anum#, cnum#), la clé est le couple, pas chaque champ séparément.

SQL regroupé en deux familles : LDD et LMD

Famille	Syntaxe / mots-clés	Description
LDD	CREATE DATABASE / CREATE TABLE	Créer une base ou une structure.
LDD	ALTER TABLE … ADD / MODIFY / DROP …	Modifier une structure.
LDD	DROP TABLE / DROP DATABASE	Supprimer une structure et ses données : prudence.
LMD	INSERT INTO t (colonnes) VALUES (valeurs);	Insérer des lignes.
LMD	UPDATE t SET champ = valeur WHERE condition;	Modifier les lignes sélectionnées.

Complément — fonctions dates et nombres utilisées dans les fiches
Dates : NOW() · YEAR(d) · MONTH(d) · DAY(d) · HOUR(t) · MINUTE(t) · SECOND(t) · DATE(d)
DATE_ADD(d, INTERVAL n unité) · DATE_SUB(d, INTERVAL n unité)
DATEDIFF(d1, d2) · Nombres : ROUND(x[, d]) · ABS(x) · MOD(a, b)

Famille	Syntaxe / mots-clés	Description
LMD	DELETE FROM t WHERE condition;	Supprimer les lignes sélectionnées.
LMD — interrogation	SELECT [DISTINCT] colonnes FROM table	Lire les données ; SELECT est parfois classé séparément comme langage d'interrogation.
LMD — filtres	WHERE … ; AND / OR / NOT ; IS NULL	Filtrer les lignes avant tout regroupement.
LMD — filtres	BETWEEN / IN / LIKE	Tester intervalle inclusif, liste, motif.
LMD — tri	ORDER BY colonne ASC / DESC	Trier le résultat.
LMD — jointure	FROM table1, table2 WHERE table1.cle = table2.cle	Relier les tables par les conditions dans WHERE.
LMD — agrégats	COUNT / SUM / AVG / MIN / MAX	Calculer sur un ensemble de lignes.
LMD — groupes	GROUP BY … HAVING …	Former les groupes puis filtrer les agrégats.
LMD — sous-requête	WHERE x > (SELECT AVG(x) FROM t)	Comparer à une valeur calculée.
LMD — absence	NOT IN (SELECT cle FROM t WHERE cle IS NOT NULL)	Rechercher les absences sans valeur NULL dans la liste.

Types, contraintes et commandes d'accompagnement

Catégorie	Éléments	Repères
Entiers	TINYINT, INT ; éventuellement UNSIGNED	Compter ou identifier ; conserver téléphone et CIN comme chaînes.
Décimaux	DECIMAL(p,s), NUMERIC	Valeurs décimales exactes ; p chiffres au total et s décimales.
Approchés	FLOAT, DOUBLE	Nombres approchés ; éviter pour des montants exacts.
Texte	CHAR(n), VARCHAR(n), TEXT	Texte ; taille et comparaison dépendent aussi de l'encodage/collation.
Binaire	BLOB	Données binaires.
Date / heure	DATE, TIME, DATETIME, TIMESTAMP, YEAR	Dates, heures ou instants ; tenir compte des différences de fuseau.
Contraintes	PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK, DEFAULT	Intégrité, références et valeurs par défaut.
Utilitaires	USE base; SHOW TABLES; DESCRIBE table;	Choisir une base et examiner sa structure ; ce sont des commandes d'accompagnement.

Complément — fonctions chaînes utilisées dans les fiches
LEFT(ch, n) · RIGHT(ch, n) · LENGTH(ch) · UPPER(ch) · LOWER(ch)
CONCAT(a, b, …) · SUBSTRING(ch, d[, n]) · REPLACE(ch, ancien, nouveau)

PHP procédural — fonctions et syntaxes

Fonctions réellement appelées ou explicitement étudiées dans le livre. […] désigne un paramètre facultatif. Les constructions echo, isset, empty, unset, include et require sont signalées comme telles. L'instruction barrée exclue du cours n'est pas ajoutée aux fonctions à employer.

Syntaxe	Description
abs($n)	Valeur absolue.
sqrt($n)	Racine carrée ; domaine réel non négatif.
round($n, $precision)	Arrondir une valeur numérique.
rand($min, $max)	Entier pseudo-aléatoire, bornes incluses ; pas pour un secret.
floor($n)	Entier inférieur ou égal.
ceil($n)	Entier supérieur ou égal.
min($a, $b, …)	Minimum des valeurs.
max($a, $b, …)	Maximum des valeurs.
gettype($v)	Nom du type.
settype($v, "integer")	Changer le type de la variable.
define("NOM", $valeur)	Définir une constante.
isset($v)	Vérifier présence et valeur non null ; construction du langage.
empty($v)	Tester une valeur vide au sens PHP ; construction.
unset($v)	Supprimer une variable ; construction.
is_string($v)	Tester si la valeur est une chaîne.
is_numeric($v)	Tester une représentation numérique ; ne garantit pas un entier.
is_finite($n)	Refuser une valeur numérique infinie ou non définie.
is_numeric($texte)	Tester si la valeur représente un nombre (entier ou décimal).
count($tableau)	Nombre d'éléments.
array_keys($tableau)	Tableau des clés, pour un parcours indexé.
array($v1, $v2, …)	Construction d'un tableau ; alternative à […].
sort($t, SORT_NUMERIC)	Trier les valeurs croissantes et réindexer.

Syntaxe	Description
rsort($t)	Trier les valeurs décroissantes et réindexer.
asort($t)	Trier les valeurs croissantes en conservant les clés.
arsort($t)	Trier les valeurs décroissantes en conservant les clés.
ksort($t)	Trier les clés croissantes.
krsort($t)	Trier les clés décroissantes.
strlen($texte)	Longueur en octets.
substr($texte, $debut, $longueur)	Extraire une sous-chaîne ; indices à partir de 0.
trim($texte)	Retirer les blancs aux extrémités.
strpos($texte, $cherche)	Position de la première occurrence, ou false ; 0 est valide.
str_replace($cherche, $remplace, $texte)	Remplacer les occurrences.
strtolower($texte)	Minuscules ASCII.
strtoupper($texte)	Majuscules ASCII.
strcmp($a, $b)	Comparer les chaînes : zéro signifie égalité.
strrev($texte)	Inverser les octets ; attention à l'UTF-8.
explode($separateur, $texte)	Découper une chaîne en tableau.
implode($separateur, $tableau)	Assembler les éléments en une chaîne.
chr($code)	Caractère correspondant à un octet.
ord($texte)	Valeur du premier octet.
strval($v)	Convertir une valeur en chaîne.
checkdate($mois, $jour, $annee)	Vérifier la validité civile d'une date.
date($format [, $timestamp])	Formater une date ; Y/y, m/n, d/j, H/i/s sont sensibles à la casse.
getdate([$timestamp])	Lire les composantes locales dans un tableau.
time()	Timestamp Unix courant, en secondes.
mktime($h, $min, $sec, $mois, $jour, $annee)	Construire un timestamp local ; normalise les dépassements.
strtotime($expression [, $base])	Analyser une date ou une expression relative ; tester false.

Syntaxe	Description
mysqli_connect($serveur, $user, $mdp [, $base])	Ouvrir la connexion et éventuellement choisir la base.
mysqli_select_db($cnx, $base)	Choisir ou changer la base.
mysqli_query($cnx, $sql)	Exécuter une requête fixe ou avec entier validé ; résultat SELECT ou true pour une modification réussie.
mysqli_fetch_assoc($res)	Lire une ligne avec noms de colonnes ; null à la fin.
mysqli_fetch_row($res)	Lire une ligne avec indices numériques ; null à la fin.
mysqli_num_rows($res)	Nombre de lignes du résultat SELECT.
mysqli_affected_rows($cnx)	Nombre de lignes affectées par la dernière modification.
mysqli_insert_id($cnx)	Dernier identifiant AUTO_INCREMENT généré sur la connexion.
mysqli_close($cnx)	Fermer la connexion.
calculerMoyenne($somme, $effectif)	Fonction du cours : moyenne, ou null si l’effectif est invalide.
echo($expression)	Afficher ; construction du langage, pas une fonction.
die ($message)	Arrêter le script avec un message ; construction.
include "fichier.php";	Charger un fichier ; avertissement si introuvable, le script peut continuer.
require "fichier.php";	Charger un fichier indispensable ; arrêt si le chargement échoue.
include_once "fichier.php";	Ne pas charger deux fois le même fichier.
require_once "fichier.php";	Chargement indispensable, une seule fois.

Conventions et limites

Élément	Règle du cours
(int), (float), (string), (bool)	Convertir explicitement ; convertir n’est pas valider.
for / while / do…while	Boucles classiques ; array_keys permet d’indexer les clés d’un tableau associatif.
$_GET, $_POST, $_SERVER, $_FILES	Superglobales ; tester présence, type et domaine des données externes.
Fonctions MySQLi	Uniquement la liste procédurale retenue ; les exemples n’enregistrent pas de texte libre de formulaire.
Erreurs de serveur	La configuration MySQLi peut interrompre le script ; un test de false ne gère pas tous les échecs.

Élément	Règle du cours
Affichage simplifié	Pas de réaffichage brut de texte reçu ; exemples fixes ou sorties numériques contrôlées.

SCANNER OU CLIQUER